University of Liverpool
Department of Computer Science
COMP702 MSc Computer Science Project

Kodro

An Offline, Grounded Platform for Designing Robots and Testing Their Behaviour in Disclosed-Fidelity Simulation

---

Vaibhav Lalwani
Student ID: 201979723

Supervisor: Keith Dures

Submitted in partial fulfilment of the requirements for the degree of
Master of Science in Computer Science

11 September 2026

Word count excludes the title page, contents, references and appendices.

Declaration

I confirm that this dissertation is my own work and that it has not been submitted, in whole or in part, for any other degree or qualification. Where I have drawn on the work or ideas of others, the source is acknowledged in the text and listed in the references. The software described here was built over the project period and is held in a version controlled repository. Product claims in this edition were checked against the repository state tagged `v2.0-submission`, so every figure quoted here can be reproduced by checking out that tag and re-running the named harness. Figures carried forward from the earlier verification run of 18 July 2026 are identified as such where they appear.

Use of generative artificial intelligence. Anthropic Claude and OpenAI Codex assisted with software implementation, debugging, test design, code review, prose revision and document formatting. Their outputs were treated as proposals rather than evidence. I selected the project direction, reviewed the resulting changes and remain responsible for the submitted software and text. Automated assistants also contributed to the simulated persona inspections reported later, which are labelled as synthetic and are not presented as human evidence. Numerical claims are retained only where a committed artefact or repeatable command supports them. No result from a human study is reported because that study has not been run.

Ethics. The reported evaluation uses simulated personas and automated traces only, with no human participants and no personal data used as research data (ethics data category A0, participant category 2). The product can store learner names, programs and attainment records locally, so those records must be treated as personal data by a classroom operator even though the default path sends them nowhere. Any later study with real users will be agreed with my supervisor and run under appropriate informed consent before it begins.

Abstract

A person with an idea for a robot needs a cheap way to test the idea before buying hardware. Kodro is an offline-first desktop and static web proving ground for that early design stage. A builder imports a robot specification containing measured motor, battery, body and sensor values, then watches a disclosed-fidelity model of that design in a simulated world. A heavier build accelerates more slowly and drains its battery faster, a stronger motor raises the closed-form top speed, and the available programming commands follow the fitted hardware. The same platform lets a non-expert assemble a catalogue robot and program it in Python or blocks. The result is a design aid and learning environment, not a same-to-same digital twin, electrical design tool or certificate that a physical robot is safe.

The honesty is structural. Every performance figure is badged HONOURED, APPROXIMATED or NOT SIMULATED, and a per-robot report states how each value was derived. JavaScript and Python implementations of the shared closed forms are held together by a constants hash, golden traces and formula-parity tests. Four declarative Prove contracts run over five controlled seeds each and emit canonical manifests containing the source hash, contract, seed, engine identity, conditions, metrics and verdict. Repeating a seed produces byte-identical evidence, and an intentionally broken controller fails the same gate. The assistant prefers a small local model through Ollama and receives the active build's command set. Grounding does not make generation infallible: the model can still invent or misuse a call. Kodro repairs when it can and otherwise withholds the program with the exact validation error, including when the requested sensor is absent. A deterministic fallback keeps the core usable without a model. The model may explain evidence, but it cannot alter a contract verdict. Reflection and skill stores are implemented as system memory, but this evaluation does not demonstrate that they reduce the iterations needed to reach a working design.

At the evaluated source state, tagged `v2.0-submission` in the repository, the complete declared Python matrix collects 1,296 tests and passes every one that its host can run, at 87.80 percent branch-aware coverage against an 85 percent gate; one or two Tk-dependent tests skip where no display toolkit is present, which is a property of the host and not of the suite; the interpreter harness passes 180 of 180 checks. Four Prove contracts pass all twenty seeded runs, reproduce byte-identically and reject a broken controller. The shipped web build passes five of five boot and privacy checks, 61 contrast and responsive checks, and the split browser matrix of six rendered flows, 34 behaviours, six layouts and 13 modal surfaces. Two gates speak directly to whether the product can teach rather than merely run. Every one of the eighteen lessons ships a worked solution, and a gate runs all eighteen through *both* marking engines on every change, requiring 100 out of 100 in each, within the constructs that lesson declares and within its own line budget; a lesson whose own answer fails does not ship. A second gate holds authored lessons to the identical criterion dispatch as the bundled ones, in 56 checks covering the document format, its validator and its on-device store. A broader world sweep passes 61 of 61 combinations and identity checks. A focused live-Ollama check compiles and runs eight of eight prompts through the real interpreter, while the larger synthetic-persona artefact completes its 40 cells. Neither is human-usability evidence. Renderer measurements, taken as three samples per tier on each rasteriser, meet the 4.17 ms render-work budget a 240 Hz display would require in every sample on the host's ordinary integrated GPU (median 144.5 FPS Low, 128.2 High), and miss it under forced software rasterisation (median 34.4 FPS Low, 28.4 High), which is the disclosed floor for machines with no working graphics driver; no universal frame-rate claim is made because the display and browser cap what is shown. These are engineering verification results, not evidence that the simulation predicts a physical robot to deployment tolerance or that real users find the

product useful. No human study is reported. The contribution is the integration of specification-driven behaviour, disclosed fidelity, build-aware generation and deterministic rejection into a zero-cost early design and learning tool.

Acknowledgements

I thank my supervisor, Keith Dures, for steady guidance and for keeping the scope honest. I thank the maintainers of the open-source libraries this work stands on, named in the implementation chapter. The framing of the project owes a debt to a long line of work on learning by building, on grounding language models in what a system can actually do, and on closing the gap between simulation and reality, all of which is acknowledged in the references.

Contents

Chapter 1

Introduction

1.1 Context and motivation

A robot is one of the few pieces of software that can hurt someone if it is wrong. A spreadsheet that miscalculates wastes time. A control program that misjudges a distance drives a machine into a wall, or into a person. This is the reason the gap between an idea for a robot and a working robot is wide, and it is why the gap is expensive to cross. Parts cost money, wiring takes time and care, and the first contact between a freshly written control program and the physical world is usually a small disaster. People who are not professional roboticists, the makers, the hobbyists, the students, the curious, feel this barrier most sharply. They have the idea and the willingness, and they lack a safe and cheap place to try the idea before they commit money and hardware to it.

The obvious answer is simulation. Try the robot in software first, watch it behave, fix what is wrong, and only then build it. This is sound, and it is exactly what professional roboticists do with heavyweight simulators. The difficulty is that those simulators are heavyweight in every sense. They assume a workstation, a graphics card, an internet connection for assets and accounts, and an operator who already knows robotics. They are built for the expert who is refining a known design, not for the non-expert who is exploring whether an idea is worth pursuing at all. A person who has never wired a board cannot sit down in front of an industrial simulator and get useful work done in an afternoon.

There is a second answer that has become tempting in the last few years, which is to ask a large language model to write the control code. This works until it does not, and the way it fails is dangerous in this setting. A language model produces plausible text, and plausible control code that calls a sensor the robot does not have, or that assumes a capability the hardware lacks, reads exactly like correct control code to a beginner. The output looks authoritative and is sometimes wrong, and the person least able to tell the difference is the very person the tool is meant to help.

There is a third failure that the first two share, and it is the one Kodro is named against. A simulator that reports a tidy number without saying how much to trust it invites the same misplaced confidence as a hallucinating model. A pretty run that quietly runs a slow robot faster than it can really go, or reports a top speed it never actually simulates, is a lie told in the language of engineering. The response is not to hide the approximations but to badge them: to say plainly, figure by figure, which numbers the simulation honours, which it approximates, and which it merely reports without ever driving.

Kodro is built where these observations meet. It is a proving ground for reducing uncertainty before purchase: a place where a skeptical builder imports a robot specification and inspects how that design behaves under a disclosed first-order model on one ordinary laptop, with no account or subscription. It treats the language model as a useful but untrusted assistant rather than an oracle, provides the active command set as grounding, and places deterministic validation between generated code and the editor. It treats its own physics the same way, disclosing the fidelity of every reported figure rather than presenting an approximation as a certified result. The platform also serves the learner who has only an idea and no specification through a parts catalogue, lessons and a design studio.

1.2 Problem statement

The problem this project addresses can be stated in one sentence. How can a free, offline-first application let a builder import a robot specification, or a non-expert design one from a catalogue, and test that design in an agent-populated simulation before buying hardware, while stating the fidelity of every figure and rejecting generated code that exceeds the active build's capabilities?

Four words in that sentence carry most of the weight. *Realistic* means visually and behaviourally legible within a stated model, not physically identical to reality. The robot must respond to its build and share the world with moving agents, while a run reports a spread rather than one tidy outcome. *Honest* means that every figure is labelled honoured, approximated or not simulated. *Offline-first* means that all core design, lesson, simulation and deterministic checking paths work with no account, paid interface or remote service. Local Ollama is optional, and the user may explicitly choose a cloud model, which is a connected mode rather than part of the offline guarantee. *Safe* means safe to inspect and test in the simulated workflow: generated code is withheld when validation fails. It does not mean that Kodro certifies software for direct deployment on physical hardware.

1.3 Aims and objectives

The aim of the project is a working offline proving ground on which a builder imports a real robot specification, or a non-expert designs one, and validates that behaviour in a realistic, agent-populated simulation before building it, with every reported figure carried at an honestly stated level of fidelity. Each objective below carries a plain test of done, so progress is judged rather than asserted.

1.3.1 Core objectives

These define the artefact. The platform must achieve all of them to count as a working system.

O1. **Specification-driven behaviour, imported or designed.** Let the user import a real robot specification of measured numbers, or assemble a build from a parts catalogue, where the specification drives the simulation. *Done when* changing the build changes behaviour: a heavier robot accelerates more slowly and drains its battery faster, a stronger motor raises the closed-form top speed, and a fitted sensor unlocks its command while an absent one withholds it from every way of programming the robot.

O2. **Honest fidelity disclosure.** Report every performance figure at one of three disclosed levels, honoured, approximated, or not simulated, and never present an approximation as a certified result. *Done when* each figure carries a fidelity badge in the Lab, the realism dashboard and a per-robot verification report, and a value the simulation does not actually drive is never badged honoured.

O3. **Safe execution and scoring.** Execute the robot's real program in a sandbox that blocks file, network and process access, and score every run against the scenario's success criteria. *Done when* a hostile program cannot reach the disk and a correct one is scored correctly.

O4. **Realistic, randomised validation.** Run the robot in a world with terrain and moving agents, across repeated randomised runs. *Done when* one program runs over several seeds and returns a report of successes, collisions, time to goal and battery used.

O5. **Grounded local assistance with a guaranteed fallback.** Prefer a local model grounded in the robot's specification, and fall back to deterministic rule-based behaviour when no model is present. *Done when* the platform remains usable with the network off and no model installed, and generated code that calls a capability the build lacks is not offered for application.

O6. **Refinement from use without retraining.** Retrieve accumulated reflections and reusable skills to inform later suggestions. *Done when*, in an ablation over repeated related tasks,

enabling memory lowers the median iterations to a working design. *Status at the evaluated build:* storage and retrieval are implemented, but the causal efficacy criterion has not been demonstrated.

1.3.2 Secondary objectives

These raise the quality of the artefact once the core holds.

O7. **Two ways to program, one meaning.** Provide a text editor and a blocks palette that compiles to the same program, so that the way the user expresses a behaviour does not change what the behaviour means.

O8. **One shared motion model.** Drive the simulation from a single set of closed-form equations and constants, mirrored across the two engines and locked together in continuous integration, so the visible studio and the tested Python engine cannot silently disagree about how a robot moves. *Done when* a constants hash, a golden-trace corpus and a formula-parity test all gate the two engines on every push.

O9. **Legible realism on a normal laptop.** Render the worlds, the named mission sites and the robot with enough fidelity to read as believable in a screenshot, while staying smooth on a machine with no discrete graphics card.

O10. **An honest record.** Build the system as small, independently tested changes behind a continuous integration gate, and keep a decision log and a test-evidence log, so the work is reproducible and the dissertation can be checked against it.

1.3.3 Out of scope

Two things are deliberately not attempted, and saying so keeps the claims honest. Kodro does not aim at production-grade physical accuracy. Its motion is a believable kinematic model, not a rigid-body solver, and the discussion chapter is explicit about what that does and does not buy. Kodro also does not aim to replace the established heavyweight simulators such as Isaac Sim, Gazebo, Webots or MuJoCo. The useful relationship with those tools is a research bridge on the roadmap, not a claim of equivalence.

1.4 Contributions

This project contributes a complete, working artefact and the engineering record behind it. Concretely it delivers:

- a desktop application, offline by default, that imports a real robot specification of measured numbers, or lets a non-expert assemble a build from a parts catalogue, and derives the robot's mass, closed-form top speed, battery life and the exact set of commands it can run.
- a three-tier fidelity disclosure, honoured, approximated and not simulated, surfaced as per-figure badges in the Lab, in a realism dashboard and in a per-robot verification report a builder can keep, so the tool never dresses an approximation as a certified result.
- one shared motion model, a single set of closed-form equations and constants mirrored in JavaScript and Python and locked together by a constants hash, a golden-trace corpus and a formula-parity test, so the visible studio and the tested engine cannot silently drift.
- two ways to program one robot, a Python subset interpreter and a blocks palette that compiles to the same Python, both driving a single command surface so the meaning of a behaviour does not depend on how it was written.
- a build-aware assistant and automatic code reviewer that receive the fitted command set, normalise known call styles and withhold invalid output after bounded repair attempts, backed by a deterministic path when no model is installed.
- a validation layer that runs a program in a sandboxed world of seventeen named mission sites

among moving agents and reports what happened, with an experience memory of reflections and reusable skills available to later sessions without retraining any model.
- a grounding metric and a seeded benchmark harness, KodroBench, that measure by syntax-tree analysis whether a generated program stays inside the build's fitted-command set, with a leaderboard generated from a machine-readable run record rather than typed by hand.
- a disciplined development process, recorded in a decision log and a running test-evidence log and gated by continuous integration, that makes the work reproducible and gives the critical reflection in this dissertation a factual basis.

The contribution is not a new algorithm. Retrieval grounding, a sandboxed gate, experience memory, skill libraries and domain randomisation are each known. The contribution is an implemented combination of specification import, three-tier fidelity disclosure, shared closed forms, build-aware generation and deterministic rejection in one early design and learning tool. The work demonstrates the mechanism of system memory, but not a causal improvement from that memory.

1.5 Report structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature the design follows from, across learning by building, the gap between simulation and reality, and the grounding and honest framing of language models. Chapter 3 sets out the requirements, the intended users including who the tool is not for, and the analysis that shaped the design. Chapter 4 describes the layered architecture and the key design decisions, chief among them the robot specification as the single source of truth and the deterministic gate that stands between a generated program and the world. Chapter 5 describes the implementation against the real code, the techniques that recur, and the engineering challenges that were met and fixed. Chapter 6 reports the evaluation, separating what was measured from what is planned. Chapter 7 states the limitations, Chapter 8 addresses professional issues and ethics, and Chapter 9 concludes. The references and appendices follow.

Chapter 2

Background and Related Work

This chapter situates Kodro against three lines of work rather than placing it beside them. The design is a consequence of these lines, so the chapter reads each one for what it implies for an offline design and validation platform, and ends by naming the niche the surveyed work leaves open.

2.1 Learning by building

The oldest of the three lines is constructionism, the durable finding that people learn most deeply when they build something external that they can watch behave and then debug (Papert, 1980). The important word is *watch*. A learner who can see the consequence of a change, and can change it again, is in a tight feedback loop that abstract instruction cannot match. Kodro is built around exactly this loop. The user assembles a robot, writes a behaviour, runs it, sees the robot behave in a world, and adjusts. The blocks palette compiles to the same Python the text editor runs, so the move from blocks to text is a change of surface and not a change of tool, and the learner is never asked to throw away what they understood in order to progress. This is a deliberate echo of the constructionist position that the medium should not get in the way of the idea.

2.2 Trusting a simulator: the gap between simulation and reality

The second line is the question of why a simulator is worth trusting at all. The central and well-documented problem in robotics is the gap between simulation and reality, the drop in performance when a behaviour tuned in simulation meets the physical world. The accepted response is not the pursuit of a perfect simulator, which is unattainable, but domain randomisation: vary the friction, the mass, the sensor noise and the scene across many runs, so that a behaviour which survives the spread is likely to survive reality too (Tobin et al., 2017). The insight is that variability is the asset, not the enemy. A program that only works on one tidy layout has learned the layout. A program that works across a randomised spread has learned something more general.

Kodro adopts the variability principle at a lightweight, pre-build level. A design is exercised across randomised seeds with varied friction, mass and sensor noise, and a run reports the spread rather than only the average. The worlds include traffic, pedestrians and other moving agents, so behaviour is tested among dynamic obstacles rather than on an empty plane. This is robustness testing within Kodro's first-order model, not evidence of transfer to reality.

2.3 The model: hallucination, grounding, and honest refinement

The third line concerns the language model itself, and it justifies both the offline choice and the careful framing of refinement. Two findings shape the design. The first is that hallucination is intrinsic to these models rather than a defect that a future version will simply remove (Huang et al., 2025). In generated code this surfaces as plausible but wrong calls (Lee et al., 2025), which is precisely the dangerous failure when the output is control code that a beginner cannot evaluate. The second is the standard mitigation: ground generation in retrieved, authoritative material rather than in the model's memory (Lewis et al., 2020). For robots, the strongest form of grounding is to constrain the model to compose a known, safe set of capabilities rather than

to invent them, in the spirit of programs written against a fixed robot interface (Liang et al., 2023; dos Santos et al., 2026) and of grounding instructions in what the robot can actually do (Ahn et al., 2022).

Kodro supplies the active command set with each generation request, but prompt grounding alone cannot prevent a model from inventing a missing sensor call. The implementation therefore validates the returned syntax tree against that set, attempts bounded repair and withholds the code if validation still fails. The grounding material lives on the machine, so this path can use a local model with no cloud. Genuine weight-level self-evolution remains an open problem (Tao et al., 2024). Kodro does not claim it. Its system memory follows verbal reflection (Shinn et al., 2023) and reusable behaviour libraries (Wang et al., 2023), with recent work offering stronger experience-driven versions of this pattern (Wu et al., 2025). The existence of these mechanisms does not establish efficacy in Kodro, which requires the planned ablation. Deterministic checks retain final authority because multi-agent arrangements have characteristic failure modes (Cemri et al., 2025). Automated review may identify suspect code (Sun et al., 2025), while deterministic syntax-tree analysis provides the enforceable rejection boundary (Khati et al., 2026).

2.4 Small local models

A fourth, narrower body of work makes the local option practical rather than merely principled. Locally deployable models paired with retrieval have shown feasibility on institutional hardware (Reza et al., 2025), and onboard small models are being investigated for robot task planning and code generation (Wang et al., 2026). These results motivate a local path. They do not establish that every one to four billion parameter model is reliable on Kodro's command surface. The evaluated model is therefore reported by task outcome, and the design relies on validation rather than assumed capability.

2.5 The niche

The reviewed work leaves a narrower integration gap. Professional simulators prioritise high-fidelity modelling and expert extensibility, while educational environments prioritise accessible programming against supported or preconfigured robots. Within the products reviewed for this project, none combined a user-entered physical specification, explicit per-figure fidelity labels, lessons, build-aware code generation and deterministic rejection in an offline-first interface. This is a bounded market observation, not proof that no product anywhere has overlapping features. The nearest evaluation harness in the robot-programming literature, RoboEval with its CodeBotler programs (Hu et al., 2023), checks generated code against one fixed robot interface. KodroBench instead measures an invented symbol against an interface that changes with the build.

Official product documentation supports the segmentation rather than the stronger claim that alternatives are deficient. Gazebo describes high-fidelity physics, rendering and sensor models with plugin and messaging entry points (Gazebo, 2026). Webots describes a desktop simulator for modelling, programming and validating robots and expects some robotics and programming knowledge (Webots, 2026). VEXcode VR deliberately provides a preconfigured virtual robot in a browser (VEX Robotics, 2026), while Open Roberta centres drag-and-drop programming of supported robots and boards (Open Roberta, 2026). Kodro's differentiator is therefore the particular workflow it integrates: start with a catalogue or measured specification, expose the resulting command set, test code in a lightweight world, and label what the model does not simulate. Optional cloud keys are a connected mode. They do not form part of the offline guarantee.

There is a second argument for the offline design, and it is stronger than the privacy and cost arguments usually offered for it. A hosted educational tool depends on somebody continuing to pay for the hosting, and that dependency has a documented failure mode. Trinket, the

embedded Python widget used across a great many United Kingdom Key Stage 3 and GCSE teaching resources, announced that it will shut down on 31 August 2026 (Trinket, 2026). The announcement is explicit that publishing the source does not preserve the service: the code is open, the site still goes dark, every teaching resource that embedded a trinket loses its interactive element, and every pupil loses saved work not exported before the deadline. That date falls eleven days before this dissertation is due.

The point is not that one company withdrew a free tier. It is that the architecture decides whether a teaching tool can be withdrawn at all. A tool whose lessons, marking, world and pupil record live in files on the machine cannot be switched off by a third party, cannot lose a pupil's work to an account deletion, and cannot be broken by a change to an interface it does not call. That property is not a side effect of the zero-cost constraint; it is the reason the constraint is worth accepting. It also means the honest comparison with a hosted competitor is not feature against feature but expected lifetime against expected lifetime, and this project makes no claim to win the former.

Chapter 3

Requirements and Analysis

This chapter turns the aims of Chapter 1 into requirements that the design can be measured against. It begins with the intended user, because the user decides almost everything that follows, and it is deliberate about who the tool is not for. It then states the functional and non-functional requirements, and analyses the central tension that shaped the whole system, which is the relationship between a helpful model and a trustworthy result.

3.1 The intended user, and who it is not for

Kodro is built for two related users. The first is the skeptical early-stage builder who wants to compare a robot idea before committing money and hardware. That user imports measured values and reads the fidelity report as a record of model assumptions, distinguishing figures the simulation honours from approximations and omissions. The second is the capable non-expert who has an idea but no lab, kit or robotics training. This may be a maker exploring a companion robot, a student testing vehicle logic or a hobbyist comparing rover designs. That user can assemble a catalogue build, learn the command surface and test behaviour on the laptop already available. Kodro helps both users form and reject early hypotheses. It does not tell either user that the physical design is proven.

It is equally important to say who the tool is not for, because a tool that claims everyone as its user serves no one. Kodro is not for a professional roboticist refining a known design to deployment tolerances. That person needs rigid-body physics and the established heavyweight simulators, and Kodro does not pretend to compete with them. It is not a gamified coding toy, and it is not a general-purpose environment for adults who already know robotics. Its third and now largest user is a school pupil working through the lesson path, and that has to be stated plainly because it changed during the project: the lesson library, the marking, the hint and worked-answer sequence and the reading-age handling were all built for that user, and they now constitute most of the product. The eighteen lessons span Key Stage 1 to Key Stage 4 but are weighted to the upper end, with one lesson at KS1, two at KS2, eight at KS3 and seven at KS4, so "Key Stage 1 to 4" is accurate only in the narrow sense that a lesson exists at each stage. The honest description of the taught range is upper Key Stage 2 through Key Stage 4, with a Key Stage 1 taster. It is not for a team that needs to collaborate over a network on a shared design. The offline, single-machine constraint that makes Kodro safe and private also makes it deliberately solitary. It is not for anyone who needs the model to be brilliant, because the model is small, local and grounded on purpose, and the design never asks it to be clever. Naming these non-users is not modesty. It is the discipline that keeps the rest of the requirements honest.

3.2 Functional requirements

The functional requirements follow the core objectives. They are stated as testable capabilities.

3.3 Non-functional requirements

The non-functional requirements are where the project's character lives, because the offline constraint touches every one of them.

- **Offline by default.** No account, no paid interface, and no data leaving the machine on any default path: the only networked call the core makes is to a local model server on the

loopback address, and the platform is fully functional with that absent. A user may explicitly opt in to connect their own cloud model key for a stronger assistant, in which case only their prompt is sent, and only to the provider they choose. The offline default is never weakened and requires no such connection.

- **Runs on a mainstream laptop.** The worlds and the robot must render smoothly on a machine with no discrete graphics card, which bounds the visual approach to procedural geometry and a measured lighting model rather than heavyweight assets.
- **Deterministic where it matters.** The simulation, the scoring and the safety checks must be reproducible, so they can be tested without a display and so a result can be trusted to mean the same thing twice.
- **Safe by default.** The system must fail toward safety. An absent model, a missing audio device, a headless environment, or a hostile program must each produce a clear, contained outcome rather than a crash or an unguarded action.
- **Legible.** A non-expert must be able to read the program surface and understand it, which is why the command API is a small set of plainly named functions rather than an object hierarchy.

3.4 The central analysis: a helpful model against a trustworthy result

The hardest requirement to satisfy is the tension between F9, helpful model assistance, and F6, safe execution. Helpfulness pushes toward flexible generation. Trust requires rejecting anything that exceeds the defined surface. A naive design exposes plausible but wrong control code to a beginner. An overly rigid design provides little help. Kodro separates generation from acceptance so each can have a different authority.

The analysis that shaped Kodro separates the two concerns by authority. The model may draft, suggest and explain, but it does not decide that a program is acceptable. Deterministic machinery owns that boundary: a fitted-command validator rejects unavailable calls, the sandbox blocks unsafe operations before execution, and the simulation gate runs and scores a program before any hardware is involved. The automatic reviewer is advisory. Its rewrite is also rejected if validation fails. The model proposes and the deterministic layer disposes.

ID	Requirement	Test of satisfaction
F1	Import or assemble a robot	The user imports a real specification of measured numbers, or picks a chassis, board, sensors and actuators, and the build derives mass, top speed, battery capacity and the set of runnable commands.
F2	Specification drives behaviour	Changing the build changes the simulation: a heavier robot accelerates more slowly and drains battery faster. A stronger motor lifts the closed-form top speed.
F3	Fidelity disclosure	Every performance figure carries a fidelity tier of honoured, approximated or not simulated, in the Lab, the realism dashboard and a per-robot verification report. A value the sim does not drive is never badged honoured.
F4	Command gating	A command appears only if the part it needs is fitted, and disappears from the editor, the blocks palette and the assistant when the part is removed.
F5	Two ways to program	A text editor and a blocks palette that compiles to the same program, both driving one command surface.
F6	Safe execution	A program runs in a sandbox that blocks file, network and process access, and a hostile program cannot reach the disk.
F7	Scenario scoring	A run is scored against the scenario success criteria, returning a pass or fail, a score, and a per-criterion reason.
F8	Randomised validation	One program runs across several seeds with varied friction, mass and sensor noise, returning a report of successes, collisions, time and battery.
F9	Grounded assistant	The model receives the fitted-command set. Generated code is checked against it and is not offered for application when a required capability is absent.
F10	Deterministic fallback	With no model installed, a rule engine returns code and checks within a fixed time budget, and the platform stays fully usable.
F11	Self-refinement memory	After a run the system records a reflection and may save a skill, and retrieves similar past material to inform later suggestions.

Table 3.1. Functional requirements and their tests of satisfaction.

Chapter 4

Design and Architecture

This chapter sets out the design that the requirements imply. Every claim here corresponds to code in the repository, and the rationale for the non-obvious choices is recorded contemporaneously in the project's decision log. The chapter moves from the overall shape down to the parts that carry the most weight: the robot specification as the single source of truth, the trace-driven core that makes everything testable, the safety machinery, and the grounded assistant with its deterministic fallback.

4.1 Design goals

Four goals shaped the architecture, in priority order. First, **legibility**: a non-expert must be able to read the program surface and understand it. Second, **determinism and testability**: the simulation, scoring and safety checks must be reproducible and testable without a display. Third, **strict offline operation**: no network dependency anywhere on the critical path. Fourth, **separation of the user's program from the engine internals**, so the user works against a tiny, safe surface and the engine can change underneath without breaking their mental model.

4.2 Technology choices and their rationale

The platform is written in Python 3.12 behind a desktop web view, and each choice follows from the offline, low-resource constraint rather than from fashion. The interface is authored in React but precompiled to a single bundle, so the panelled layout and the modern look are kept while the cost of a runtime compiler is paid once at build time and never on the user's machine. The three-dimensional world uses a copy of the Three.js library held inside the project, core only, so the graphics work with the network off and with no asset pipeline. The user's designs and the system's accumulated record are stored in SQLite, a single standard-library file with no server, so a backup is a file copy and nothing leaves the machine, and a whole project, the build, the programs, the world and the accumulated memory, saves to and loads from a single self-contained `.kodro` file. The assistant calls a local model served by Ollama, which installs as one small application and keeps a model warm with a single command a non-technical user can follow. The desktop shell is pywebview, which renders the vendored interface in a native window using the platform web view, with a Tkinter interface retained as a guaranteed-offline fallback for machines without a modern web view runtime.

4.3 Layered architecture

The system is organised in layers with a one-directional dependency flow, so that each layer depends only on the ones below it. Figure 4.1 shows the design surface where this begins, the Robot Lab, and the layering is as follows.

4.4 A less-is-more interaction model

The primary journey has three stages: Design, Prove and Build. Simple is the default for a first time or occasional user. Design contains the robot choices and a single next action. Prove opens a mission cockpit rather than the programming environment. It presents the robot, world and test as three plain choices, computes a deterministic readiness judgement before the run, and exposes one Run this test action rather than duplicating Run in the global toolbar. The same

Layer	What it is	Where it lives
Program surface	The Python subset and the blocks palette	`interpreter.js`, blocks palette
Command registry	The single set of commands the build allows	derived from the robot specification
Robot specification	The single source of truth, imported or designed	`RobotLab.jsx`, `specschema.js`, the derive step
Shared motion model	The closed forms and constants both engines obey	`motion-model.js`, `engine/motion_model.py`
Fidelity disclosure	The three-tier badges and verification report	`specschema.js`, `realism.jsx`, `verify.jsx`
Worlds and motion	Worlds, named mission sites, the natural-motion tick	`Viewport3D.jsx`, `terrains.jsx`, `agents.jsx`
Validation and memory	Scoring, randomised runs, reflections, skills	`memory.jsx`, the Python grader
Python engine	The metres world, physics wrapper, public API	`engine/`, `rover_api.py`

Table 4.1. The layered architecture and where each layer lives in the codebase.

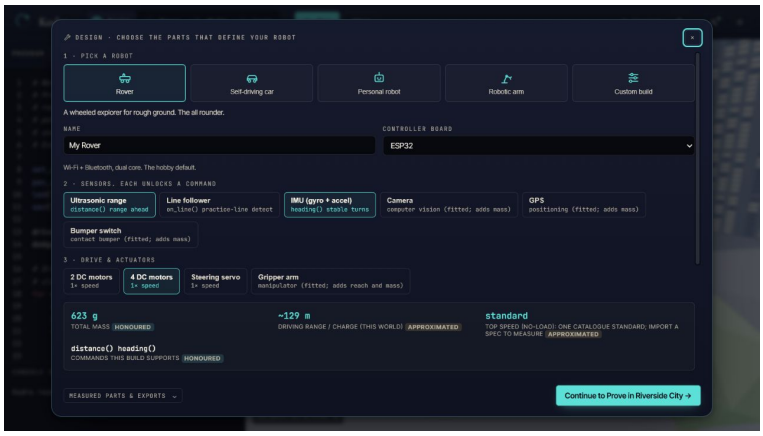

Figure 4.1. The redesigned Robot Lab in the default Simple experience. Robot type, board, sensors and actuators remain available in one focused surface, while fidelity badges and the next Design-to-Prove action stay visible without exposing every instrument at once.

surface changes from preparation to live, paused and completed states, with distance, remaining battery and closest obstacle shown after an exact run. A stored result is displayed only when its robot, world and source program match the current plan, which prevents stale evidence from being mistaken for the latest test. Build turns the active design and evidence into an explicitly provisional build brief. Editing is a deliberate transition through Edit program, and the local Companion remains visible as the route to assisted construction. Lessons, teacher progress, replay, blocks, simulation limits and project history sit behind one More Tools control instead of competing with the primary stages. Expert is an explicit setting that restores the persistent editor, console, evidence rail and denser controls for a user who needs them. The selected level is stored locally. Nothing is removed from the underlying product, so an expert does not lose capability and a beginner is not forced to understand every panel before the first run. At mobile widths the same three stages become a fixed bottom navigation, and the secondary tools remain available through the same More Tools route.

4.4.1 Input, process and output by stage

The three-stage shell is an input-process-output model made visible to the user. Table 4.2 records the contract of each stage and prevents a presentation-only redesign from obscuring what the system actually does.

Stage	Input	Process	Output
Design	Catalogue choices or an imported measured specification	Derive mass, motion values, fitted capabilities and fidelity tiers	Active robot specification, command set and disclosed assumptions
Prove	Active build, program, world, contract and seed	Validate commands, run the deterministic engine and score immutable criteria	Seeded manifest, PASS or FAIL verdict, metrics and simulation-boundary report
Build	Active specification and the latest matching evidence	Summarise requirements and distinguish honoured, approximated and unverified values	Provisional build brief for competent datasheet and safety review

Table 4.2. The input, process and output contract of the three primary stages.

4.5 The robot specification as the single source of truth

The most important design decision in Kodro is that the robot is not cosmetic. Its specification is a single object that the rest of the system reads, and it is the reason changing the build changes the behaviour. The specification arrives one of two ways. A non-expert assembles it in the Robot Lab, where a derive step computes from the chosen parts a structured build: the chassis type, the board, the fitted sensors and actuators, and from those a mass in kilograms, a top speed, an acceleration, a turn rate, a battery capacity and a drain rate, a friction profile, and crucially a list of supported commands. A builder with a real machine instead imports a Kodro Robot Specification, a small JSON document validated by a schema in `specschema.js`. It carries the total mass, the body footprint in centimetres, the drive geometry (wheel radius, wheelbase, motor count and per-motor no-load rpm, stall torque and nominal voltage), the battery capacity, voltage and usable fraction, and the sensors with their mount position, yaw and range. The schema drops unknown keys with a warning, clamps mildly out-of-range values visibly rather than silently, and rejects wild ones, and it round-trips: a build can be exported back out as a specification for a peer to import. From that point the imported numbers, not a catalogue proxy, drive the simulation.

The command list is what makes the design honest. The registry is generated from the specification rather than written independently per panel. A build without an ultrasonic sensor has no distance-reading command in the registry, so the editor rejects it and the blocks palette does not offer it. The model is told the same registry, but may still emit an unavailable call. Validation then withholds that output. This distinction between reducing an error through grounding and enforcing the boundary through deterministic rejection is central to the safety claim.

The physical fields drive the motion. For an imported build the top speed follows a closed form from the motor rpm and wheel radius, the tractive force from the stall torque and motor count, the acceleration and braking from that force against the mass, and the battery runtime from the capacity against the drawn power. For a catalogue build the same character is derived from the chosen parts. Either way a heavier robot has a lower acceleration and a faster battery drain, a stronger motor raises the top speed, and the surface traction changes how the robot holds the ground. The point is that these are not separate sliders the user tunes. They fall out of the specification, so the design loop is to change the robot, not to change a number, and to watch the behaviour change as a consequence.

4.6 The three-tier fidelity disclosure

A proving ground earns trust by being candid about its own limits, so the second load-bearing design decision is that every figure the simulation reports carries a disclosed level of fidelity. Three tiers do the work. A figure is **honoured** when the simulation actually runs the measured number: the commanded distances and turn angles, the collision circle sized from the body footprint, the command availability gated on fitted parts, the battery treated as a hard budget the robot halts at, and, when a build's motor-derived top speed falls inside the simulable band, that top speed itself. A figure is **approximated** when a first-order model stands in for the real physics: acceleration and braking as a trapezoid rather than an integration of forces, turn timing from track geometry rather than wheel torque curves, traction as three coarse bands per surface, and battery drain as a constant-power ledger with no voltage sag or thermal derating. A figure is **not simulated** when it is reported for reference but never actually driven: slopes and terrain height, wheel-level slip and per-motor torque curves, suspension and three-dimensional contact, the water, depth pressure and vacuum of the underwater and space sites (which change gravity, traction and light only, so a clean run there is scoped to the surface and never claims the build would survive the hazard itself), and the command semantics of the parts that carry no runnable command yet.

The discipline of this scheme is that it is enforced at the edges, not asserted in prose. A top speed that falls outside the band the simulation can run is a case worth dwelling on, because it is exactly where an honest tool is tempted to cheat. A cheap educational rover with a slow motor has a real top speed below the simulable floor. Rather than run it at its true crawl, the simulation runs it at the floor, which is faster than real. The honest response, and the one the code takes, is to refuse to badge that number honoured: the badge drops to approximated and the warning states the real direction, that the top speed is below the floor and is simulated faster, rather than papering over the substitution. The same holds at the ceiling for a fast build. This case matters because it is the precise failure the whole product exists to prevent, a distorted number shown under a reassuring badge, and the design closes it by making the badge follow the truth of what the simulation runs. The tiers are surfaced three ways: as per-figure badges beside the live specification in the Lab, as a full table in a realism dashboard, and as a per-robot verification report, a standalone document that lays out the build as simulated with each figure tagged by tier and by how it was derived, for a skeptical builder to keep and check.

4.7 One shared motion model

The third design decision answers a question the first two raise. If the imported specification drives the simulation, and the fidelity badges promise that a figure is honoured, then the number the user sees in the visible studio and the number a test asserts in the Python engine must be the same number, or the honesty is a fiction. Early in the project they were not the same. Four hand-rolled kinematic replicas, one in the JavaScript studio, one in the Python engine, one in the scenario validator and one in the self-test, drifted apart until they disagreed by an order of magnitude on how much battery a metre of travel costs. The design response was to collapse them into one shared motion model: a single set of closed-form equations and named constants, expressed once and mirrored in two files, `motion-model.js` for the studio and `engine/motion_model.py` for the Python engine.

The model is locked at three levels, each a gate in continuous integration. The constant table is hash-gated: both files serialise their constants to a canonical form and a test fails if the two hashes differ, so a constant cannot be changed in one engine without the other. The catalogue-mode behaviour is golden-trace-gated: a corpus of programs is run through both engines and their displacement, distance, heading and battery are asserted to agree, so a change that alters how a robot moves fails unless both engines change together. Most recently, the physical-mode closed forms are formula-parity-gated: the twelve closed forms plus the sensor-pose transform

were ported from the studio's JavaScript into the Python engine, and a test runs the studio model over a reference robot and fails on any drift. An imported build's derived numbers, its top speed, stall force, mobility, acceleration, energy, runtime, slope, turn timing, stopping distance and sensor pose, are therefore provably identical across the two engines. The honest scope of this parity is stated plainly in the limitations chapter: the two engines share the formulas, but the full end-to-end simulation of an imported build, in particular the sensor rays that account for a sensor's mount pose and range, runs only in the studio, while the Python engine reproduces the derived numbers and grades catalogue-mode motion. The design goal was never to claim identical simulators. It was to make the numbers the fidelity badges rest on impossible to fake in one engine and not the other.

4.8 The trace-driven core

A single abstraction underpins scoring, hinting and replay: the trace. Every call the running program makes to the command surface appends an event that records the call, its arguments, its result, a snapshot of the robot's state, and any collision. From this one trace three features are derived without duplicating the representation of what the robot did. Scoring compares aggregates of the trace, the samples collected, the collisions, the battery used, the distance, the final position, against the scenario's declared success criteria, and an analysis of the source against the allowed constructs, to yield a pass or fail, a score, and a per-criterion reason. Hinting runs a set of rule predicates over the same events plus the score and surfaces the first that matches. Replay reconstructs the run from the serialised trace and scrubs through it.

The value of this choice is that the decision logic is pure. The scorer is a function from a scenario, a trace and the source to a result, with no input or output and no hidden state, which makes it trivially testable with synthetic traces and impossible to make non-deterministic by accident. The same purity holds for the hint rules and the report. Side-effecting wrappers, the file writes and the dialogs, are thin shells over these pure cores. This is why the system can be tested thoroughly without a display, which the evaluation chapter relies on.

4.9 Safety: the sandbox, the executor, and the gate

The user's program is untrusted, and the design treats it that way at three points. Before execution, the sandbox walks the program's syntax tree and rejects unsafe imports and builtins, the file opens, the process spawns, the attribute walks that reach for forbidden capabilities. During execution, the executor runs the program in a daemon thread under a hard wall-clock timeout, folding every failure mode, a syntax error, a runtime error, a sandbox violation, a timeout, into one structured result rather than letting an exception escape. After execution, the simulation gate is the run itself: the program drives the robot in a world and is scored, so a wrong program fails visibly on screen against the scenario rather than silently on real hardware.

These three points are the concrete form of the central analysis from Chapter 3. The model never touches any of them. It cannot relax the sandbox, it cannot extend the timeout, and it cannot decide that a program passed. The deterministic layer owns safety end to end, which is what lets the model be helpful without being dangerous.

4.10 Worlds, mission sites, and moving agents

The validation has to happen somewhere believable, so the design recommends a world to fit the robot. A self-driving car is dropped into the City, with one-way lane traffic that loops and brakes for the robot and pedestrians that keep to the pavements and use the crossing. A home or arm robot is dropped into the Room, with furniture and people. A rover is dropped onto a planetary terrain. Beyond these base worlds sit seventeen named mission sites, real places given their own treatment: the Sahara, the Amazon, Antarctica, the Thar Desert in India, the Maasai

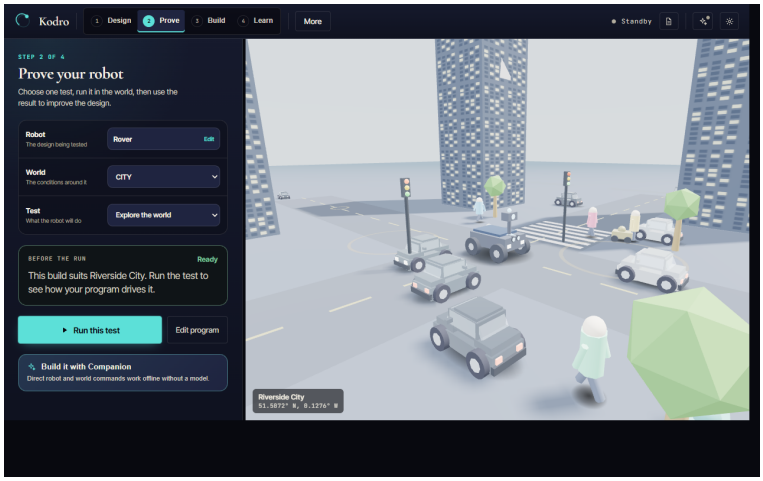

Figure 4.2. The default Simple proving surface. The mission cockpit exposes only the robot, world, test, deterministic readiness check and one Run this test action beside the three dimensional City. Program editing and the evidence history are deliberate transitions rather than permanent panels.

Mara, the slopes of Mount Fuji, the Giza plateau, an Icelandic lava field, the Himalayan foothills, the Great Barrier Reef, the Mariana Trench, Olympus Mons on Mars, the Moon's Tycho crater, Europa's ice crust, a robotics test bay, a warehouse and a minimal debug grid. Each carries its own traction, lighting and atmosphere, so the same program behaves differently across them and a run reads as a place rather than a plane. The open worlds are alive: traffic keeps to one-way lanes and eases rather than stepping, pedestrians walk their pavements and a few cross at the crossing, and a small autonomous fleet of other robots roams the open terrains, picking fresh goals and steering around the obstacles, each other and the user's robot, so the scene has machines to share it with rather than being an empty stage. Figure 4.2 shows the City world in three dimensions, which the user can orbit a full turn to read the scene from any angle.

The motion is designed to read as natural rather than to be physically exact. The robot does not slide like a prop. It transfers weight under acceleration, banks into turns, settles on its suspension, and steers from the front wheels where the chassis has them. The honest description of this, returned to in Chapter 7, is that it is a per-type kinematic model that fakes the feel convincingly rather than a rigid-body solver that resolves forces. This is a deliberate trade. The believable feel is what a non-expert needs to judge a behaviour, and it runs smoothly on a laptop, where a full solver would not.

4.11 The grounded assistant and its deterministic fallback

The assistant is designed around the principle that the model may generate but validators decide. It receives the robot specification and command registry, normalises known compatibility call styles and validates the resulting syntax. When a request needs a part the build lacks, Kodro returns the missing-capability error and exposes no Apply action. Bounded repair is allowed, but a failed repair remains a refusal rather than a plausible program. The automatic reviewer follows the same rule: a proposed rewrite is accepted only after validation, otherwise the original is retained with the rejection reason.

Behind all of this is the deterministic fallback, which is what makes the offline guarantee real. If no model is installed, or if the model does not answer within a fixed time budget, a rule engine returns rule-based code and checks, and the platform stays fully usable. The fallback is not a degraded mode bolted on. It is the floor the whole assistant stands on, and the design treats the model as an optional accelerator above a guaranteed deterministic base.

4.12 Self-refinement without retraining

The final design element is a memory loop. Three local, countable mechanisms carry it. After a run the system stores a reflection and can retrieve related notes for a later suggestion. Validated routines can be saved as named skills, and past designs remain available for inspection. None of this edits model weights. It is system-level retrieval, and calling it an improvement would require the planned controlled ablation. The present evaluation demonstrates storage, retrieval and display, not a fall in iterations.

Chapter 5

Implementation

This chapter describes how the design was realised. It is grounded in the codebase and in the running test-evidence log, and the engineering challenges are drawn from the real commit history, which makes them honest material for critical reflection rather than a tidy story told after the fact.

5.1 The program surface and the interpreter

The user's program runs through a Python subset interpreter, written in JavaScript and vendored into the bundle, which compiles a source string into a generator that yields motion and sensor events one at a time. The generator design is what lets the studio animate a run smoothly: the application pumps the generator, advancing the robot one event per tick, so the user sees the robot move rather than jumping to a final state. The interpreter supports the constructs a non-expert needs, sequence, selection, iteration with both `for` and `while`, functions, and the sensor reads, and it guards the pathological cases a beginner can stumble into, such as an enormous exponent, so that a slip produces a contained result rather than a hang.

A subtlety worth recording is that two call styles are both valid and both correct. A bare call such as `move_forward(2)` moves two metres, scaled internally to engine units, while a method call such as `rover.forward(200)` moves two hundred centimetres. This dual surface is convenient but it is also a known source of the arena-scale mismatch discussed in Chapter 7, and it is recorded honestly rather than hidden.

5.2 The robot specification and the derive step

The Robot Lab implements the single source of truth from the design. A catalogue of boards, sensors, actuators and chassis types backs a derive step that computes the structured specification from the chosen parts. The derive step is where the physical character of the robot is set: the mass is summed from the parts, the top speed and acceleration follow from the motor and the mass, the battery capacity and drain follow from the board and the load, and the supported-commands list is assembled from exactly the parts that provide each command. Saving a build dispatches an event carrying the recommended world, which the studio listens for, so designing a robot and being dropped into the right world to test it are one action.

5.3 Importing a real robot: the KRS schema

Alongside the catalogue sits the import path, which is what makes Kodro a proving ground rather than only a studio. The Kodro Robot Specification is a JSON document with a schema and a validator in `specschema.js`. A builder presses Import spec, the desktop bridge or a file dialog reads the document, and the validator returns a cleaned specification with a list of warnings and errors surfaced in the Lab. The validation is deliberately forgiving where it can be and strict where it must be: an unknown key is dropped with a warning rather than rejected, a value mildly outside its band is clamped and the clamp is shown, and a value wildly out of range is refused outright, so a hand-written or third-party specification imports usefully without silently becoming a different robot. When a specification carries motor and drive geometry, the physical closed forms take over from the catalogue proxies: the derived object carries a `phys` block whose numbers, not a parts factor, drive the simulation. The same schema exports, so a build assembled

in the Lab or imported and adjusted can be written back out as a specification for a peer, and the export carries the derived numbers alongside the inputs so the receiving side can check them.

5.4 The shared motion model and its conformance gates

The one shared motion model of the design is two mirrored files. `motion-model.js` holds the constants and the closed forms for the studio. `engine/motion_model.py` holds the twin for the Python engine. The physical closed forms are the load-bearing part: the top speed as $v = (\mathrm{rpm}/60)\,2\pi r$, the tractive force as motor count times stall torque over wheel radius, the acceleration as that force less rolling resistance over mass, the battery drain from a rolling-resistance power model against the usable energy, the differential turn timing from track geometry and the Ackermann turn radius from wheelbase and steering angle, the stopping distance as $v^2/(2\mu g)$, and the sensor-mount transform that places a sensor's reading at its offset and yaw. Three tests gate the pair on every push. A conformance test serialises both constant tables to a canonical form and compares their SHA-256 hashes, so a constant edited in one engine and not the other fails the build. A golden-trace test runs a corpus of programs through both engines and asserts their displacement, distance, heading and battery agree, so catalogue-mode motion cannot drift. A physical-golden-trace test runs the studio's closed forms over a reference robot through a small Node fixture and asserts the Python twin returns identical values to a relative tolerance of 10^{-12}, so the derived numbers an imported build shows are provably the same on both sides. The honest boundary, recorded in the limitations chapter, is that the Python engine's sensor rays still originate at the rover centre with fixed module ranges and do not yet consume an imported sensor's mount pose or range, and the public Python API does not import a specification's mass, rpm or battery, so a measured build is simulated end to end only in the studio while the Python engine mirrors the formulas and grades catalogue-mode motion.

5.5 The verification report

The fidelity disclosure is implemented as pure functions in `verify.jsx` that turn the active build into a prediction sheet and join it with the evidence the app already holds, the last live run's odometer and wall-clock and the last multi-seed validation report. Every row carries its figure, its value, its fidelity tier and a one-line statement of how it was derived, for example that a top speed came from $v = (\mathrm{rpm}/60)\,2\pi r$ on the imported motor rather than from a catalogue anchor. The report renders to a standalone HTML file a builder can save and keep, and because the functions are plain JavaScript with no React they are exercised headless by the QA harness. Nothing in the report computes new simulation state. Every number is either a motion-model closed form or a recorded result, which is what lets the report make its honesty claim truthfully.

5.6 The worlds and the natural-motion tick

The three-dimensional 3D Viewport is built on core Three.js with no asset loader, so every robot, obstacle, terrain feature and agent is procedural geometry generated in code, boxes, cylinders, spheres, extruded shapes and canvas textures, lit by an environment map with shadow mapping and tone mapping. The open-terrain ground is shaded rather than flat: a tileable normal map, derived by a Sobel pass over a periodic height field, and a roughness map are both generated in canvas at scene-build time, so the sun and fill light graze real micro-relief instead of a coloured plane. The City and the Room are built in code, and the moving agents, the traffic and the pedestrians, run in a shared coordinate space with one-way lanes, looping paths, and a rule that brakes them for the robot. The natural-motion tick is the code that gives each robot type its feel: the per-type pitch under acceleration, the roll into a turn, the suspension settle, and the front-wheel steer, applied as a function of the robot's velocity and heading change each frame. The world is chosen from a catalogue of presets, the City and the Room alongside a Robotics Lab, a Warehouse, a Minimal Debug Grid and a set of real outdoor terrains, each carrying its

own friction and lighting so the same program behaves differently across them. A render-quality control spanning Low to Cinematic bounds the two heaviest costs, the shadow resolution and the pixel ratio, so the scene stays smooth on a laptop with no discrete graphics card on the lower tiers and maxes its fidelity for a screenshot on the highest. The Cinematic tier additionally runs a small, hand-written, fully offline post chain, a bloom built from a bright-pass and a separable Gaussian blur composited additively over the base render together with a vignette, written directly against the core renderer because the library ships its post-processing only in an examples tree that is not vendored and cannot be fetched under the offline rule. The pass is gated so that it is disabled under a reduced-motion preference, turned off if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its targets, so the offline and low-hardware guarantees hold on every tier. A first-person toggle drops the camera onto the robot for a driver view.

5.7 Offline shading without an asset pipeline

The offline rule shapes the renderer as sharply as it shapes the model. The single largest lever for visual quality in a real-time scene is usually an asset pipeline: a glTF or GLB loader and a set of authored, well shaded models. That lever is closed here. Three.js ships its loaders and its post-processing only in an examples tree that is not part of the vendored core, and the zero-network constraint means neither the loader nor any quality interchange model can be fetched and committed. A hand-authored model would in any case be no better than the procedural geometry it would replace. The honest design response is to push the procedural surfaces as far as they will go and to be candid that an importer is future work rather than a missing feature that was skipped.

Two pieces of that work are worth recording because they were done under the constraint rather than around it. The first is surface relief. The open-terrain ground already carried a baked colour grain, but light still struck it as if it were glass, which was the strongest tell that the world was a tech demo. Each ground now generates, in canvas at scene-build time, a tileable normal map together with a roughness map. The normal map is a Sobel pass over a periodic height field whose waves are integer multiples of the tile size, so the derived normals are seamless under repeat wrapping. The result is that the existing physically based sun and fill light graze real micro-relief, and sand reads as sand, regolith as pits, the seabed as ripples. No texture file is loaded and no new binary is vendored.

The second is a post-processing pass for the highest quality tier. Because the library's bloom pass is not available offline, the pass is written directly against the core renderer: the proven base image is drawn to the canvas exactly as the lower tiers draw it, the scene is then rendered once more into an offscreen target used only as a bloom source, and a bright-pass, a separable Gaussian blur, an additive bloom composite and a vignette are layered on top. Drawing the bloom additively over the unchanged base, rather than replacing the image, means the worst case is that the scene simply looks like the normal render, which makes the feature safe to ship. It is gated to the Cinematic tier, disabled under a reduced-motion preference, turned off automatically if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its render targets or if the pass throws at frame time. The shading and the post pass are headless safe: with no document or WebGL context the generators return nothing and the scene renders exactly as before, so the offline bundle-render test never touches a canvas. This is the offline constraint treated as a design input, not an apology.

5.8 Memory, the assistant, and the fallback

The memory store records reflections and saved skills and retrieves them before selected suggestions. The assistant can call a local Ollama server, lists the installed models and lets the user override the automatic model choice. It passes the active command set through both genera-

tion and review, normalises supported call styles, validates the syntax tree and fails closed after bounded repair. The same contract is implemented in the static browser build, where a hosted page may contact the user's loopback Ollama server only when that server permits the page origin. The deterministic rule engine remains the guaranteed path when a model is missing or unavailable. An explicit bring-your-own-key provider is also available. That connected mode transmits prompts to the selected provider and is outside the offline guarantee.

5.9 Engineering discipline

Each unit of work closed only when a full gate passed: the test suite with branch coverage, the linter, the format check and a strict type check, after which the change was committed, pushed, and verified green on continuous integration. The coverage gate ratcheted upward as the codebase matured and settled at a minimum of eighty five percent line and branch, which every push must clear to merge. This discipline is itself a contribution, because the decision log and the test-evidence log it produced give the claims in this dissertation an audit trail that can be checked against the repository.

5.10 Representative listing

The scorer illustrates the trace-driven, pure-core principle from the design. It takes a scenario, a trace and the source, and returns a structured result with no input, no output and no hidden state, which is exactly what makes it testable with synthetic traces.

```
@dataclass(frozen=True, slots=True)
class GradeResult:
    passed: bool
    reasons: list[str]    # one user-facing line per failed criterion
    score: int            # 0 to 100

def grade(lesson: Lesson, tracer: Tracer, source: str = "") -> GradeResult:
    aggregates = _compute_aggregates(tracer.events())
    reasons: list[str] = []
    for criterion in lesson.success_criteria:
        reason = _check_criterion(criterion, aggregates, lesson, source)
        if reason is not None:
            reasons.append(reason)
    score = max(0, 100 - SCORE_PENALTY_PER_FAILURE * len(reasons))
    return GradeResult(passed=not reasons, reasons=reasons, score=score)
```

5.11 Showcases, icons, and project files

Three showcase programs ship as example tabs and double as living documentation of the command surface, each verified to run in the interpreter QA sweep. Encore is the flagship: functions, loops, nested loops, counters and pen geometry over the base command set alone, so it runs on every build and shows the language working. Searchlight is a battery-managed survey and rescue mission, a robot given real work, running a systems check and patrolling a search area with every drive guarded by the distance sensor so it never touches a wall. Gauntlet is a capability reference that draws its own proof, exercising recursion, nested loops, chained comparisons, list building and a run of pen geometry. Around these, the interface chrome was professionalised: a procedural SVG icon set replaces every emoji-as-icon, each icon a handful of primitives built offline against `currentColor` so it renders identically on every operating system, which the interpreter QA enforces by asserting zero emoji glyphs remain in the chrome source. A whole project, the build, the programs, the active world, the render settings and the accumulated memory, serialises to and from a single `.kodro` JSON file with bounded sizes and a validator that never throws, so a session is portable and a backup is a file copy.

5.12 Removing the voice route

An earlier build offered a third way to program, a voice route with speech-to-text and a spoken assistant. It was removed in full: the voice agent, the microphone input, the text-to-speech, the voice settings and the Python voice modules are gone, and a search of the shipped web source finds no speech recognition, no microphone access and no voice-to-code path. The visual `say` command remains, but it is a console line the robot writes, not a spoken output and not a voice input. The decision was a scope one made honestly: the voice path could not be made reliable and cross-platform under the offline constraint without an optional component the design did not want to depend on, so rather than ship a fragile feature it was cut, and this dissertation describes the two ways to program that actually ship.

5.13 Two front-ends over one engine

The interface exists in two forms over the same engine. The original is a Tkinter interface that ships with Python and renders on any machine, retained as the guaranteed-offline fallback. After formative review found the Tkinter look dated for an audience that benchmarks software against web and mobile apps, a second front-end was added in the medium the reference design was authored in, a vendored offline React interface rendered in a pywebview window. Both front-ends drive the same engine, scorer, library and store. The honest risk here, recorded in the next chapter, is the dual interpreter: the web interface ships its own JavaScript interpreter for instant animation, whose surface differs from the Python API, and the planned convergence is to stream real engine events to the view and retire the second interpreter.

5.14 Challenges met and fixed

These are real defects, found and fixed, and they make the strongest reflection points.

- **A procedural surface over a stateful engine.** A flat set of free functions has to reach a single live engine. This was solved with a module-level binding that the functions consult at call time, which also let the headless tests and the scorer drive the engine with no interface present.
- **An open loop.** An early build ran the user's program but never responded to it, with no pass or fail and no feedback. The fix wired scoring, hinting, persistence and the verdict into a single finish step at the end of every run, which a structured review had identified as the single biggest weakness of that build.
- **A silently detached trace.** While fixing the loop, the active trace could be detached between runs, so a run recorded zero events and some integration tests passed vacuously. The fix re-asserts the trace and engine bindings at the start of every world reset, and the tests were strengthened to prove real movement, since the battery only drains if the robot actually moved.
- **Four kinematic replicas that had drifted.** The studio, the Python engine, the scenario validator and the self-test each carried their own copy of the motion arithmetic, and they had drifted until they disagreed by an order of magnitude on battery cost per metre. The fix collapsed them onto the one shared motion model and locked it with the constants-hash, golden-trace and formula-parity gates, so the number the fidelity badges rest on cannot differ between engines again.
- **Two honesty defects in the fidelity surfaces.** A panel review of the version 2.0 build found the two most damaging kind of defect for a proving ground, a sim that tells the user a false thing about a real build. A slow imported robot was simulated faster than its real top speed while still badged honoured, with warning copy that named the wrong direction. The predictive design check used a different mobility model from the live simulation, so the two headline surfaces contradicted each other on the shipped reference robot. Both were

fixed: the top-speed badge now drops to approximated and states the true direction when the speed falls outside the simulable band, and the design check drives its mobility verdict from the same force-ratio model the simulation uses when an imported build is present. These are recorded here rather than buried because they are the clearest evidence that the honesty discipline is enforced by review and repair, not merely asserted.

- **Headless interface fragility.** A modal error dialog blocked forever on the headless test runners, once burning a job to the six-hour ceiling. A per-test timeout was added so any hang fails fast with a traceback, which then pinpointed further headless issues and led to treating one platform's interface tests as informational while the other two remain hard gates.

5.15 Packaging

The desktop binary is built with PyInstaller, which bundles the interpreter, the runtime, the libraries and the asset bundle into a single windowed application, so deployment is one download with no separate runtime install. The build is reproducible from a specification file and was smoke-tested on a real desktop, where the packaged application starts cleanly and renders the full interface, which the headless suite otherwise exercises without a display.

5.16 Web deployment

The desktop application remains the full-featured path, but the same vendored interface now also ships as a zero-install web build. A flag on the bundle build script emits a fully self-contained static site, plain files with no server-side component, which a continuous-integration workflow deploys to GitHub Pages, so the studio can be tried from a link without installing anything. The offline character survives the move in two ways. First, the build is a progressive web application: a manifest and a service worker precache the application shell, so after the first load the site keeps working with the network off. The service worker serves same-origin requests cache-first and never touches a cross-origin request, so a call to a user's own optional cloud key or to a localhost Ollama server can never be intercepted, cached or leaked by the caching layer. Second, the privacy claim is enforced by a gate rather than asserted: a dedicated check builds the static site, serves it from the loopback address, loads it in a headless browser under a network log, and fails if the default page makes a single request to any external host, alongside a proof that the application actually boots.

What runs where is stated precisely because the two builds have different persistence boundaries. The simulation core is client-side JavaScript in both: the interpreter, randomised validation, worlds, three-dimensional viewport, Robot Lab, verification report, memory and assistant provider layer. The lesson library is generated from the desktop source and parity checked. The browser now grades lesson attempts through a JavaScript grader, supplies progressive hints and stores a combined concept-strength record in that browser. The desktop bridge retains separate pupil identities and the fuller teacher record. Browser classroom evidence is therefore functional but not multi-pupil. Both modes use the same build-aware validation contract for model code.

Chapter 6

Evaluation

This chapter records how Kodro was evaluated. It separates what was actually measured, the automated verification and an analyst-run formative review, from what is planned but not yet done, a study with real users. Nothing here reports data from human subjects, because that study has deliberately been left for a point at which it can be run properly under consent.

6.1 Evaluation strategy

Three complementary methods were chosen, in increasing cost and decreasing frequency.

Method	What it checks	Status
Python test suite	Correctness of every subsystem, regression safety	Done, continuous
Interpreter QA harness	Interpreter, kinematics and product coherence end to end	Done, continuous
World and interface nets	Site identity across worlds, interface flows and modals	Done, continuous
Prove contract replay	Seed reproducibility, metric provenance and regression failure	Done, continuous
Persona review	Whole-product usability across user types	Done, formative
Persona-task evaluation	Assistant code generation and the self-test safety net, scored by execution	Done, deterministic
Renderer evidence	Observed cadence and render-submission work against a 240 Hz budget	Done, hardware and software floors measured
Adversarial persona panel	Reproducible code defects across simulated makers, verified against source	Done, seven rounds
KodroBench grounding benchmark	Invented-symbol rate and seeded task success of local models	Done, v0.1 measured
User and refinement study	Real efficacy and the refinement loop	Planned, future work

Table 6.1. The evaluation strategy and the status of each method. The last row is not yet run.

6.2 Automated verification

The primary evidence of correctness is the Python suite, which grew with the codebase and gated every change. At the state tagged `v2.0-submission`, the complete declared matrix, including the optional RL and URDF surfaces, collects 1,296 tests at 87.80 percent branch-aware coverage against an 85 percent gate, and passes every one its host can run. One or two Tk-dependent tests skip where no display toolkit is present, which is a property of the host rather than of the suite. Unit, property-based and headless integration tests pin the procedural API, deterministic physics and scoring, sandbox rejection, hard timeout, shared-motion conformance and the run,

score, hint and persistence loop. The separate interpreter harness passes 180 of 180 checks. Strict lint, format and type checks also pass, and the wheel builds successfully.

Alongside the Python suite, a separate interpreter QA harness drives the shipped interpreter with the real kinematics and the wall ray, and asserts the command semantics, every bundled example program including the three showcases, and a set of product-coherence checks. It passes at one hundred and eighty of one hundred and eighty. Every example terminates, moves, stays inside the arena, never hits a wall and never throws, and the command semantics, the two motion call styles, the turn, the speed clamp, the guarded huge exponent, the loops and the sensor reads, are each asserted, as are the interpreter diagnostics, the honesty of the design-check command list and the absence of emoji glyphs from the interface chrome. This harness is the evidence that the user-facing program surface behaves, independently of the Python engine.

The deterministic Prove gate covers four declarative contracts: straight transit, a corner route, obstacle clearance and battery reserve. Each contract is replayed over five seeds while obstacle offset, range noise, start delay and initial battery vary within declared bounds. The resulting manifest records the source SHA-256, contract identifier, seed, engine identity and source hash, sampled conditions, metrics and verdict. It deliberately has no wall-clock timestamp, so an identical source, contract and seed produce byte-identical JSON. The candidate baseline passes all twenty runs. A second replay is byte-identical, baseline comparison reports no regression, and an intentionally broken controller returns a failing process status with all four contracts failed. The human-readable companion report states that the evidence comes from a kinematic simulation and cannot certify a physical robot. The local Companion may interpret these records, but the verdict is calculated and owned by the deterministic runner.

Browser evidence is split so that a degraded launcher cannot be mistaken for a product result. The shipped static build passes five of five boot, mount and zero external request checks, and the deterministic stylesheet gate passes 61 contrast and responsive assertions over ten themes. The split headless matrix passes six of six rendered flows, 33 of 33 behaviour assertions, six of six layouts and 12 of 12 modal surfaces. Those checks cover program execution, distinct world identity, the Blocks path, error recovery, capability honesty, lessons, classroom records, replay, deterministic proof evidence, an imported physical specification and layouts down to 320 CSS pixels. The separate world sweep passes 61 of 61 checks across six worlds, six robot choices, three quality tiers, 17 mission sites and weather or time changes. At desktop, tablet and mobile widths the measured document width matched the viewport, the three primary stages did not wrap, the mobile bottom navigation remained usable and the browser error log was empty.

A final returning-user check found a persistence defect that clean-profile tests could not expose: corrected coaching text could be shadowed by obsolete prose saved inside an older run report, and a robot display name did not prove that the saved result came from the current configuration. The released fix fingerprints every persisted robot input that can affect behaviour and requires an exact world, robot name, configuration and source match before a saved run can appear as evidence for the open plan. Legacy reports remain available as history but cannot be presented as current proof. The cockpit also regenerates its coaching sentence from the structured measurements and current diagnostic rules instead of trusting persisted prose. A real run followed by a full reload retained its 13.9 m distance, 10.2 percent battery use and 67 cm clearance, while the obsolete safety wording remained absent.

These nets earned their place by catching defects the unit tests could not, because those defects live in the wiring rather than in a pure function: selecting a real-world site resolved through the wrong table and crashed the view. The three-dimensional robot mesh faced ninety degrees off its travel direction while the physics stayed correct. The parts catalogue advertised commands the interpreter does not implement. Several controls fell below the accessible-contrast and touch-target thresholds. Each confirmed finding was fixed and then re-verified against the net.

6.3 Measured renderer evidence

The evaluated build publishes a rolling 120-frame report from the real three-dimensional render loop. It separates request-animation-frame cadence from CPU-side render submission because the former is capped by the browser and virtual display while the latter is the work budget the application controls. Table 6.2 holds two committed runs from `scripts/qa_performance.mjs`, one on the host's real GPU and one with software rasterisation forced, because the two answer different questions: the hardware row is what an ordinary laptop experiences, and the software row is the guaranteed floor on a machine with no working graphics driver at all. Each run is three independent samples per tier (`-repeat=3`) reporting the median with the observed range, because a single frame-rate reading on a shared machine is not repeatable: the same build and harness produced 18.4 and 29.9 FPS in the software High tier depending only on what else the host was doing. Both artefacts are SHA-256 locked to the harness and the shipped bundle, and each run records which rasteriser class actually drew it, read from the unmasked WebGL renderer string, so a software figure cannot be presented as a hardware one or the reverse.

Rasteriser and tier	Observed FPS	P95 frame	P95 submission	4.17 ms budget
Hardware, Low	144.0 (143.2 to 144.9)	7.6 ms	2.5 ms	Met
Hardware, High	144.9 (126.8 to 145.1)	7.9 ms	3.5 ms	Met
Software, Low	33.0 (25.4 to 33.3)	34.1 ms	10.8 ms	Not met
Software, High	28.0 (19.3 to 28.9)	39.9 ms	5.6 ms	Not met

Table 6.2. Renderer evidence from Windows headless Chrome, median of three samples per tier with the observed range in brackets. Hardware rows use the host's integrated Intel GPU through ANGLE; software rows force SwiftShader. On ordinary integrated graphics the render submission work meets the 4.17 ms budget a 240 Hz display would require in every sample; under software rasterisation five of six samples miss it, and that floor is what the adaptive quality governor exists for. Neither row is a claim about displayed FPS, which the display and browser cap.

The interface exposes the same measured cadence, P95 submission time, effective quality and budget verdict in its Evidence rail, and drops to the Low tier only after two sustained miss windows. The pair of results reframes the earlier software-only reading: the application's own render work is comfortably inside the strictest budget measured on commodity hardware, and the low frame rates previously reported are a property of CPU rasterisation on driverless hosts, the disclosed worst case rather than the expected experience.

6.4 Persona review (formative)

To evaluate the whole product rather than its units, a structured review was conducted across simulated personas spanning the range of makers who might use the tool. Each persona drove a session and returned a mark out of ten with the concrete defects it found. After each round the named defects were fixed and the build was re-rated. Table 6.3 reports the measured trajectory.

Round	Personas	Focus	Mean (/10)
1	50	First contact with the build	5.86
2	12	After the first round of fixes	6.50
3	50	After wiring the model and tools into the app	7.10
4	50	After grounded answers were added	7.36
5	8	Focused review of the new three-dimensional world	6.40

Table 6.3. Persona review trajectory across rounds. The figures are reported as measured by the analyst-run method, with the threats to validity stated below.

6.5 Objective persona-task evaluation

The persona review above is a subjective inspection, and its single-analyst scoring is its weakest point. A second evaluation therefore asks a narrower reproducible question: whether the offline assistant turns plain-language requests into programs that compile, run, stay inside the arena and complete an executable task predicate. Eight simulated personas, a beginner, a younger literal reader, a teacher, a precise maker, a low-vision voice-first user, a learner using English as an additional language, a robotics engineer and a safety-minded skeptic, each attempt the same five tasks with persona-specific phrasing: drive forward, turn and move, drive a square, stop before an obstacle using the distance sensor, and use a counted loop. The local `qwen2.5-coder:3b` model answers at temperature zero with base seed 4046. The shipped interpreter and self-test, not a model judge, decide every outcome over up to three correction turns. The committed artefact retains the exact prompts, raw replies, extracted code, execution trace summaries, seeds, model digest and hashes of the harness and deterministic evaluators. Table 6.4 reports the measured funnel over 40 persona-task cells.

Outcome (over 40 persona × task cells)	Cells (%)
Compiled (valid program produced)	40/40 (100%)
Ran clean through the interpreter	40/40 (100%)
Stayed inside the arena (no wall hit)	40/40 (100%)
Completed the task	40/40 (100%)

Table 6.4. Objective persona-task funnel, local model `qwen2.5-coder:3b`, base seed 4046, temperature zero and up to three correction turns. The shipped interpreter and self-test judge every row. Mean turns to success was 1.0.

Persona group	Complete	Task	Complete
Beginner and younger learner	10/10	Forward	8/8
Teacher and maker	10/10	Turn and move	8/8
Low-vision and EAL	10/10	Square	8/8
Engineer and skeptic	10/10	Obstacle stop	8/8
		Counted loop	8/8

Table 6.5. Task completion broken down by persona and by task.

The 40/40 result establishes one useful but narrow fact: this exact quantised 3.1-billion-parameter model, prompt contract and deterministic evaluator completed these five short tasks under eight phrasing profiles in the committed run. It supersedes a historical 7/20 run made with the earlier `kodro-coder` artefact, but the difference cannot be attributed to one fix because the model, persona set and harness all changed. Nor does a perfect result on a small known task set establish open-ended code generation, learning, usability or real-robot safety. The value of the harness is that every accepted reply leaves executable evidence and a reproducible failure path; a model score can never overrule compilation, runtime, arena or task gates.

Three additional role prompts inspected the same summary as an advisory panel. The usability and robotics roles returned PASS, while the methodology role returned FAIL for the correct reason: synthetic personas are not human participants and must be treated cautiously. These roles were three prompts to the same local model, not independent experts, and their verdicts are retained only as critique. The methodology failure is therefore not averaged away. It identifies the next required evidence, a consented study with actual users, while the deterministic 40-cell funnel remains the only authoritative result in this subsection.

6.6 Adversarial code-verified persona panel

The two persona methods above ask about opinion and about task execution. A third method asks a different and more falsifiable question: can a large adversarial panel of simulated makers find real, reproducible defects in the shipped code, and does fixing what it finds converge? Unlike the formative review, nothing here is a mark out of ten, and unlike the objective persona-task evaluation, nothing here is scored by whether a generated program runs. Each persona is instead pointed at the source with a single instruction, to try to break the tool and to distrust every number it prints, and every issue it reports is then put through an adversarial verification pass that defaults to not-real and only accepts a defect that reproduces end to end against the shipped files. The whole panel is run offline as a fan-out of local analysis agents, and its output is a list of confirmed, code-cited defects rather than a score. Each confirmed defect was accepted only with a file-and-line citation that reproduced against the shipped files, and the fixes that closed the confirmed defects are recorded in the project's commit history.

The panel was run in seven rounds, each on the build the previous round's fixes produced, and the arc of the rounds matters more than any single finding. The first round put thirty personas against the v2.0 build and, after adversarial verification and de-duplication, produced four confirmed defects, all in the fidelity-badge disclosure. The strongest was a genuine honesty bug in which a top speed clamped into the simulable band was still labelled "honoured" by the live readout and the exported report, even though the schema had computed the correct, lower-confidence tier all along. Because the obvious risk with any find-and-fix pass is that it converges on the one area it happened to look at, the second round aimed twenty eight fresh personas deliberately away from the badges, at keyboard, screen-reader, contrast, offline-guarantee, import-fuzzing, determinism, mobile and reduced-motion surfaces, and it did not come back empty: it produced thirteen further confirmed defects, five of them high severity, led by a persistent boot failure in which a corrupted project file threw at start-up on every reload until browser storage was cleared by hand, alongside a keyboard focus trap and motion that ignored the reduced-motion setting. A third round widened the aperture to the Python engine and its sandbox, the two engines cross-checked formula by formula, the assistant's grounding path, the memory store and packaging. Sixteen of its thirty lenses came back empty, and the rest produced sixteen further confirmed defects, two of them correctness bugs in the grading layer itself: a grader that counted the distance a program commanded rather than the distance the rover actually travelled, so a rover clamped against a wall could pass a distance requirement it never met, and environment sensors hardcoded to Earth values, so a program that branched on gravity or temperature was graded on the wrong world. Every confirmed defect in these three rounds was fixed and the automated gates were re-run to green.

The fourth and fifth rounds swept the remaining corners with thirty personas each, from the replay dialog and the hint engine to every Tk panel and all eighteen lessons. Half of the fourth round's lenses came back clean, and its twenty-three verified findings were, with one exception, maintainability and polish rather than correctness, the exception being in-progress lesson code that lived only in memory and was lost on refresh. Twenty were fixed, two interpreter language changes were recorded as deliberate limitations of the restricted, golden-trace-gated teaching subset rather than chased, and one reported defect proved on implementation to be intended behaviour. The fifth round returned the most clean lenses yet, twenty of thirty, and most of its fifteen findings were polish, but it corrected any impression that the tail holds only trivia. One finding had been latent since the first commit: the grader was never actually enforcing the no-collisions criterion, because it counted collisions from an event stream the engine does not emit while the real count accumulates on the rover snapshot, so a program that drove into a wall passed a no-collisions lesson with full marks, and four rounds had walked past it because the only tests touching it hand-built synthetic events the engine never produces. Six further findings were deferred with stated reasons and one reported item was already correct.

The sixth round changed tactics: rather than sweep blind, it pointed a third of its thirty personas

at a single known bug class, since the round-three commanded-distance bug and the round-five collision bug shared one shape, an aggregate read from the wrong source instead of the real number the engine records. Tracing every metric the grader, hint engine, report and achievements compute back to its source found a third and a fourth copy of the same wrong-source pattern that five rounds had missed, one in the fallback application's own collision count and one in the hint engine's distance, alongside two camera animations still ungated by the reduced-motion contract and an interpreter parity gap. All eight were fixed. The seventh round audited a new feature on the day it shipped, the optional bring-your-own-key path that lets a user connect their own cloud model alongside the offline default, with sixteen personas weighted toward the security of the new surface. The security verdict was clean: no request can reach a non-local host with the default provider or with no key set, the key is never logged and is sent only to the provider the user picks, and cloud-generated code passes the same deterministic self-test gate as local code before it can reach the learner. Thirteen of the sixteen lenses came back clean, and the six confirmed findings were all honesty of labelling rather than security, panels and documentation that still claimed fully local answers while a cloud key was answering. All six were fixed the same day.

That each pass on a freshly fixed build kept surfacing real defects is the clearest evidence that this method is not a rubber stamp, and it is also a plain reminder that a passing test suite is not the same thing as a sound product. Across the seven rounds, one hundred and ninety four simulated personas found eighty five confirmed defects, of which seventy five were fixed, eight were deferred with stated reasons, and two were reclassified or found already correct. The share of adversarial lenses that came back clean rose from none in the first round to over three-quarters by the sixth and above eighty percent in the seventh, and the sixth round's clearest lesson was methodological, that hunting a known bug class directly finds instances a broad sweep walks past, the wrong-source aggregate alone turning out to have four copies scattered across the codebase. Both facts hold at once, and together they are the closest thing to a convergence signal this method can give: the code became demonstrably more trustworthy under adversarial reading, and the testing never reached a point where the product could be declared done. The honest limits are the same ones that apply to every simulated method in this chapter, and they are worth stating rather than glossing: these are language-model personas and not people, the panel measures the code and not a learner, and a clean round is evidence of nothing more than that a particular set of adversarial lenses found nothing that reproduced. It is a cheap and repeatable net that keeps catching real bugs before a human has to, which is precisely why it earns a place next to the deterministic suites and precisely why it cannot be read as validation. It does not license a perfect score, and the human study remains the right next step.

6.7 Measuring grounded code generation (KodroBench)

The methods above evaluate the product. A final method measures, in isolation and with a number, the specific failure the grounding design exists to prevent: a language model emitting a program symbol that the built robot does not expose. The distinction from prior art is worth stating carefully. The nearest neighbour is RoboEval with its CodeBotler harness (Hu et al., 2023), which scores language-model programs for service robots by executing them and checking temporal properties over the resulting traces. Its programs, however, are written against one fixed robot interface, so the characteristic hallucination it can surface is an invalid argument passed to a valid primitive, for example a navigation call to a location that does not exist. General benchmarks of interface hallucination in ordinary software measure invented calls but involve no robot and no execution of the offending program. What none of the surveyed work measures is an invented *symbol* against a *per-build* interface. Because Kodro derives the command set from the user's build, the ground truth for what counts as a valid call is per-design rather than fixed: a model that is perfectly grounded for one robot can invent for the next, and the same call is valid on one build and hallucinated on another. KodroBench was built to measure exactly that

intersection.

The metric is deliberately small and deterministic. The grounding checker parses a generated program into its abstract syntax tree and classifies each call it finds. A bare-name call is invented when it names neither a command in the build's fitted-command set nor one of the three builtins the sandbox exposes, unless the program itself defines that name or aliases it from a fitted command. An attribute call is invented when it is made on an object the program never created, since the fitted API is a set of bare functions and exposes no objects: a call such as `rover.forward()` is therefore counted, and only a real built-in container or string method on a name bound to a literal container, for example `xs.append(1)` after `xs = []`, is exempt. That attribute rule is what lets the metric catch an entire object-style command surface rather than waving it through as ordinary Python. A syntax error is recorded as a result rather than raised as a failure, and the function is pure, so the same program and fitted set always return the same verdict. Around it, the benchmark harness gives each model a natural-language instruction together with exactly the commands the build exposes, then runs the generated program across ten seeded, domain-randomised runs through the same Python engine the desktop application uses, following the randomisation argument of Tobin et al. (2017). A run succeeds when the program moves at least the task's minimum distance without a collision, and the reported success is the mean over seeds and tasks. The harness also supports the unbiased pass@k estimator of Chen et al. (2021) when several independent samples are drawn per task, and carries a development and held-out task split whose two held-out tasks are adversarial, in that the instruction names a capability the build lacks, for example a build with no distance sensor. With five tasks this split is a mechanism rather than a rigorous held-out claim, and it is reported as such. Growing the suite toward twenty to fifty tasks is what would make it meaningful. The results are written as a machine-readable record and the leaderboard is generated from that record, never typed by hand, so every number in Table 6.6 traces to a run. A deterministic floor, a fixed grounded program per task, is byte-reproducible and gives continuous integration a value to assert against.

Model	success@N	Invention rate	Syntax errors	Collisions
Deterministic floor	0.22	0.00	0.00	1.24
`gemma3:4b`	0.24	0.00	0.20	0.56
`gemma3:1b`	0.02	0.00	0.20	0.18
`llama3.2:3b`	0.00	0.00	0.00	1.44
`kodro-fast` (local fine-tune)	0.00	0.60	0.40	0.00
`kodro-coder` (local fine-tune)	0.00	1.00	0.00	0.00

Table 6.6. KodroBench v0.1 (five tasks, ten seeds per task). success@N is mean seeded task success. Invention rate is the fraction of tasks whose program calls a symbol outside the build's fitted-command set. Syntax errors and collisions are the per-task rates. Values are as recorded in the versioned results file.

The headline finding is honest and it is not the one that was hoped for. The two models that invent are the project's own local fine-tunes, produced earlier in the project specifically for the assistant role. `kodro-coder` emits an object-style `rover.forward()` call surface on every one of the five tasks. Against each task's fitted-command set, which for the benchmark's Python engine is a set of bare functions with no `rover` object, every such call falls outside the fitted set and the grounding checker flags it, giving an invention rate of 1.00 and a task success of 0.00. One honest qualification belongs here: the web studio's own interpreter additionally accepts the object-style `rover.forward()` form as a documented compatibility alias for the bare verbs, so that surface is not universally rejected across the product, and what the benchmark measures is invention against the per-build fitted set it prompts the model with, on which the object surface is ungrounded. `kodro-fast` invents on three tasks of five and drifts into unrelated prose on the other two. The general small models never invent a symbol: given the fitted-command

list in the prompt, they stay inside it, and their failures are weaker ones, malformed indentation, missing arguments, or a plausible name used as a bare statement, so their programs mostly parse but rarely complete the task. The zero collision rate of the two fine-tunes is failure rather than caution, since a program built from invented calls never drives the robot at all. The deterministic floor completing only 0.22 says the seeded obstacle field is genuinely hard for a fixed open-loop program, which keeps the model rows in proportion.

Two readings follow, and both matter to the design. First, the metric works: it caught, at scale and mechanically, a failure mode of this project's own artefacts that a compile check or an opinion score would have missed, an entire invented command surface, and the result is reported as measured precisely because it cuts against the project's own fine-tunes. Second, the benchmark localises the failure precisely: every invented call in the table is a call to a symbol outside the task's fitted-command set, which is exactly the class of mistake the grounding metric exists to name. Whether a given surface is then refused at run time depends on which engine runs it, and the honest statement is narrower than a blanket vindication: on the benchmark's Python engine the fitted set contains no `rover` object, so the object-style calls are ungrounded there, whereas the web studio interpreter accepts the `rover.forward()` alias and would run the same program. The benchmark therefore quantifies the ungrounded-symbol failure the metric was built to measure, without claiming the product refuses every such call on every surface.

The threats to validity are stated plainly. Five tasks are a proof of mechanism, not a benchmark of record. The success predicate is a coarse motion criterion, moving far enough without collision, rather than a task-semantic judgement. The runs are simulation only, on a single machine, over a handful of small local models, and the table reports a single-sample run rather than a pass@k average. The two fine-tunes were trained by the same author, so their failure is evidence about these specific artefacts, not about fine-tuning in general. What survives those limits is still useful: a deterministic, seeded measurement that needs no graphics card, runs in continuous integration, and puts a number on an axis the surveyed prior art leaves unoccupied.

6.8 Threats to validity

The persona review is a low-cost inspection method in the tradition of heuristic evaluation, and it is reported with its limits stated plainly rather than dressed up as user evidence. Three threats matter. First, a single analyst simulated every persona, so the scores carry that analyst's bias and the inter-rater reliability is unknown. Second, the personas are informed guesses about real makers and cannot capture genuine confusion, motivation or the messiness of real use. Third, re-scoring a build the analyst made risks optimism, so a rising mean should be read as a direction of travel and not as a validated usability score. The objective persona-task evaluation removes the analyst from pass/fail scoring, but it carries its own limits: its personas are still generated phrasings rather than people, its five predicates are coarse proxies, its one seeded model run is not a distribution, and repeated or related prompt patterns can reward task familiarity. The two advisory PASS verdicts are not independent because the same local model produced them; the methodology FAIL is retained precisely to prevent that panel becoming self-validation. The renderer measurement has a separate external-validity limit: SwiftShader headless timing is reproducible evidence for this host, not a proxy for a user's GPU or display. None of these methods measures learning, classroom efficacy or physical transfer, and none substitutes for the planned human study.

6.9 The planned human evaluation

One bounded pilot study is specified and left for a point at which it can be approved and run with people. Its research question is whether a deterministic evidence view helps a user diagnose a failed robot program more accurately than the raw console alone. A target of twelve participants, with ten to fifteen accepted for the pilot, completes both conditions in a counterbalanced within-

participant design. Six condition sequences are crossed with three mission-order rotations to reduce simple order effects. The three missions expose a goal shortfall, an unsafe clearance and insufficient battery reserve. The primary outcome is correct diagnosis. Time to diagnosis, confidence, perceived workload and the evidence items used are secondary measures.

The full protocol, participant information sheet, consent form, task script, measures, collection templates and standard-library analysis script are versioned under `docs/study`. The analysis computes condition summaries and paired participant differences, with deterministic bootstrap intervals and an exact sign-flip test reported as exploratory because the pilot is small. No result is entered into this dissertation until ethics approval, recruitment, consent, data collection and a supervisor-reviewed analysis have occurred. At the candidate release state the pack is marked `ETHICS_PENDING`; the study has not recruited anyone and the templates contain no participant rows. This study addresses usability of deterministic evidence. It does not validate physical transfer, classroom learning or the efficacy of the reflection memory.

6.10 Instrumentation

The application is instrumented for this study without requiring a cloud service. A run can record its program, trace, score, battery and collisions, while reflections, skills and classroom progress remain in local stores by default. Export is explicit. These records may still be personal data when they identify a learner, so the study requires data minimisation, retention rules and consent even when no default network transfer occurs.

6.11 Summary of the evaluation

The product is strongly verified against its stated software contracts, but usefulness and physical predictive validity are not established. The 1,296-test Python matrix, 180-check interpreter harness and deterministic web gates pass at the state tagged extttv2.0-submission. The four Prove contracts pass twenty seeded runs, replay byte-identically and reject the broken controller. The local-Ollama checks run cleanly, but their generated phrasings are not people. The renderer report preserves a negative result: its software-rendered host missed the 240 Hz work budget in both tiers. The adversarial synthetic panel found real defects, but its outputs remain bug-finding evidence rather than user evidence. The planned human study is ethics-pending and has not been run. No claim of memory-driven improvement, classroom efficacy, purchasing accuracy, universal frame rate or physical transfer follows.

Chapter 7

Discussion and Limitations

A dissertation that only reports what works is not trustworthy. This chapter sets out, plainly, what Kodro does not do yet, so that a reader and a future contributor can plan around the gaps. Each limitation is framed as something the design is working toward rather than something broken with no path forward, and each is labelled by honest status: working with a known gap, experimental, or roadmap.

7.1 Motion is kinematic, not rigid body

The most important limitation to be clear about is the one most easily oversold. The robot and every moving agent advance through a hand-written kinematic tick. There is no rigid-body solver, no contact forces and no friction model resolving the motion in the runtime. A car banks and a rover throws its weight around because the per-type motion code fakes the feel convincingly, not because a physics engine resolves forces, and collisions are detected as overlap events and recorded rather than resolved with impulses. This is a deliberate trade that buys a believable feel at a laptop-friendly cost, and it is the right trade for a non-expert judging a behaviour. It is not the right trade for deployment-grade validation, and the dissertation does not claim it is. The roadmap item that addresses this is the wiring of a real rigid-body engine into the production runtime.

7.2 The physics engine exists but is not on the hot path

Related to the above, the Python engine ships a tested wrapper around a real two-dimensional physics library that builds boundary walls, places the robot and obstacles, and records collisions. It is exercised by the Python suite and used by the Python scorer. It is not, however, what the visible studio runs, because the shipped Studio interface steps the world through the vendored kinematic interpreter. The rigid-body path is real code, but it is not on the hot path the user sees when they press run. Making that wrapper the source of truth for the visible motion is the single largest change on the roadmap, because it touches the interpreter, the 3D Viewport and the scorer at once.

7.3 Sensor command coverage is partial

The grounding guarantee that the design rests on is fully enforced today for the ultrasonic sensor (the distance command), the inertial unit (the heading and tilt commands) and the line follower (the on-line command), which are the commands the interpreter actually implements. The scan command is gated on the same ultrasonic part. The line follower reads a synthesized practice line rather than a painted lane in the world, and is gated on that basis. The remaining parts, the positioning sensor, the camera, the bumper and the gripper, are real fitted hardware that change the build but carry no implemented command yet, so they are deliberately not advertised as callable rather than offered as a sensor a build cannot use. This is the gap between the design's promise and the current code, and it is named here rather than glossed, because the whole safety argument depends on closing it. Full gating across every surface is a near-term roadmap item, and it is the first thing the realism work should finish.

7.4 Procedural geometry, with surface shading and a gated post pass

The 3D Viewport has no asset loader for any interchange format, so it cannot import a real robot description or a modelled part. Everything is procedural geometry. This part of the limitation is genuine and stands. The shading side, however, is now mitigated rather than absolute. Each open-terrain ground builds a tileable normal map, derived by a Sobel pass over a periodic height field, together with a roughness map, both generated in canvas at scene-build time, so the light grazes real micro-relief instead of a flat coloured plane. The Cinematic quality tier adds a small, hand-written, fully offline post chain, a bloom built from a bright-pass and a separable Gaussian blur composited additively over the base render, with a vignette. It is written directly against the core renderer because the library ships its post-processing only in an examples tree that is not vendored and cannot be fetched under the offline rule, and it is gated to the Cinematic tier, disabled under a reduced-motion preference, turned off automatically if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its targets, so the offline and low-hardware guarantees are not weakened. What remains genuinely absent is a glTF or similar asset loader and the heavier effects, ambient occlusion, depth of field and motion blur. The loader stays on the roadmap rather than shipped because neither it nor good interchange models can be obtained under the zero-network constraint in this environment, and that is recorded as future work rather than presented as a gap that was quietly skipped.

7.5 An arena-scale mismatch between the two engines

The in-browser interpreter runs the robot in a thirty-metre arena while the Python engine and scorer work in a ten-metre world. Every bundled example runs and the conformance check passes, because the criteria are scale tolerant, but a program tuned visually in the web interface can score differently under the Python scorer. The metres-to-centimetres scaling in the interpreter mitigates this for motion but not for boundary behaviour. A single shared arena definition that both engines read is the fix, and it is recorded as a known issue rather than left for a reader to discover.

7.6 The measured-build simulation is the studio only

This is the most important scope limit of the fidelity claim, and it is stated bluntly. The one shared motion model locks the two engines at three levels, the constants, the catalogue traces, and now the physical closed forms, so an imported build's derived numbers, its top speed, stall force, acceleration, energy, runtime, turn timing, stopping distance and sensor pose, are provably identical in the studio and in the Python engine. What is not yet shared is the full end-to-end simulation of an imported build. The Python engine's sensor rays still originate at the robot's centre and use fixed module ranges, so they do not consume an imported sensor's mount pose or its range, and the public Python API does not read an imported specification's mass, rpm or battery. The consequence is precise: a measured build is simulated end to end only in the JavaScript studio, while the Python engine reproduces the derived numbers and grades catalogue-mode motion. This is why the honoured sensor-pose and range figures are scoped in the fidelity table to the studio simulation, and why the badges never claim that the Python grader honours an imported sensor's geometry. Closing this, by wiring the sensor pose and range into the Python sensor model and teaching the public API to import a specification, is a defined roadmap item and the natural next step after the formula parity that already landed.

7.7 Memory exists, but improvement is not demonstrated

The product stores reflections, skills and relationships between robots, worlds and outcomes, and the retrieval paths are exercised by deterministic checks. That proves mechanism and persistence only. The objective required a lower median number of iterations with memory enabled than

with it disabled across related tasks. No such ablation has been run, so objective O6 is not complete and the title no longer presents the platform as proven self-refining.

7.8 Build guidance is not purchasing or electrical advice

The Build surface generates an evidence-bounded concept bill of materials from the active design. It names design requirements and distinguishes values honoured, approximated and not simulated. It deliberately does not provide live prices, supplier stock, wiring, logic-level compatibility, current limits, battery protection or mechanical fit, because none is verified. The exported brief repeats those warnings. An opt-in, source-cited research workflow over manufacturer data is future work. Until then a competent person must check original datasheets and electrical safety before purchase or power-up.

7.9 Frame rate is measured, not guaranteed

The three-dimensional viewport now measures its own rolling frame cadence and CPU-side render submission, exposes both in the Evidence rail and drops costly effects after sustained budget misses. That is a control mechanism, not a promise that a browser will display 240 frames per second. The monitor refresh, browser scheduler, GPU, driver, viewport, scene and competing processes remain outside the product's control. The committed SwiftShader run missed the 4.17 ms budget in both tiers, and its low observed cadence is reported as a negative host result rather than generalised to hardware rendering. A credible release claim therefore requires a device-and-browser benchmark matrix with named displays and GPUs; even that would describe tested configurations, not guarantee every machine.

7.10 The small-model ceiling

Finally, the local model is small and its output quality varies by model and prompt. Build grounding reduces off-surface suggestions but does not prevent them. The enforceable property is narrower: invalid code is rejected and not offered for application. The deterministic path, not model intelligence, carries the offline and code-acceptance guarantees.

Chapter 8

Professional Issues

This chapter separates the ethical position from the wider BCS professional issues and project criteria.

8.1 Ethics

No human participant took part in any study reported in this dissertation, and no personal data was used as research data. The evaluation personas are language-model constructs, not people. The text labels them as simulated wherever they appear, and no conclusion about real users is drawn from them. Under the department's ethics classification the reported work sits in data category A0 with participant category 2.

The default configuration uses no account, telemetry or analytics, and the shipped static application makes no external request at boot. Pupil names, programs, class records, run reports and saved skills are nevertheless personal data when linked to a learner. They are stored locally and can be cleared, but the classroom operator remains responsible for lawful use, access control, retention and deletion. Local Ollama sends prompts only to loopback. If a user explicitly configures a cloud provider, prompt content leaves the device for that provider. The interface labels this connected mode and it is outside the offline privacy claim.

Two further matters belong here. First, the Declaration names the generative systems used and assigns responsibility to the author rather than to those systems. Second, the planned study with real users will require its own ethics application, informed consent, data minimisation, a retention schedule and agreement with the supervisor before it begins.

8.2 BCS professional issues and project criteria

The BCS criteria call for analytical application of degree material, self-management, synthesis beyond routine application and critical self-evaluation. Kodro applies these through a recursive-descent interpreter, mirrored kinematics, property-based and coverage-gated testing, and local model use under retrieval grounding. Its synthesis combines specification import, fidelity disclosure, a sandboxed gate and build-aware assistance without claiming a new algorithm. Version control, three-operating-system integration and the decision records evidence self-management. Failed model outputs, incomplete objective O6, the missed performance target and absent human and physical validation are reported rather than hidden.

The principal professional safety risk is false confidence. Kodro therefore labels model fidelity, refuses invalid generated commands and states that simulation is not certification. Local-first storage and no default telemetry reduce disclosure, but an operator still has data-protection duties for identifiable classroom records. A configured cloud provider receives prompts only after explicit selection. Accessibility is gated through contrast, keyboard, touch-target, reduced-motion and narrow-viewport checks, with the planned participant study needed for issues automation cannot observe. Security controls reject file, network and process access, bound execution time and keep advisory output separate from deterministic acceptance.

Vendored and third-party work is identified. Low quality and effects downgrades reduce computation, but no environmental benefit is quantified.

Chapter 9

Conclusion and Future Work

9.1 Summary

This project set out to build a free, offline-first proving ground in which a builder imports a robot specification, or a non-expert designs one, programs it through text or blocks and tests the design in an agent-populated simulation before buying hardware. The artefact delivers specification-driven behaviour, three-tier fidelity disclosure, mirrored and conformance-gated closed forms, sandboxed execution and scoring, named mission sites, a functional classroom path, build-aware model assistance and deterministic rejection. The final interaction design defaults to a Simple three-stage Design, Prove and Build journey, retains an opt-in Expert surface and moves secondary tools behind one disclosure control. At the evaluated source state 1,205 Python tests, 180 interpreter checks, twenty seeded Prove runs and the web gates pass. An intentionally broken controller fails. The local-Ollama runs are executable evidence rather than user evidence, and the measured renderer meets the 240 Hz work budget on the host's integrated GPU while missing it under the disclosed software-rasterised floor. The reflection and skill stores work as memory mechanisms, but objective O6 remains incomplete because no controlled evaluation shows that memory reduces iterations. The simulation reduces early design uncertainty. It does not reproduce or certify a physical robot.

9.2 Reflection

Four honest reflections stand out. What worked was building scoring, hinting and replay on one trace abstraction and keeping the decision cores pure, which paid off repeatedly in testability and in the ease of adding features late. What was harder than expected was headless interface testing, where a browser launch can time out on a loaded machine, so the honest outcome was to treat the browser-based world and interface nets as a smoke signal rather than a hard gate and to lean on the deterministic interpreter QA and Python suite for the load-bearing evidence. The reflection that carries the most weight for a proving ground is that honesty is not a property you assert once but one you have to enforce and repair: a panel review of the version 2.0 build found two defects in the exact fidelity surfaces the product stakes its credibility on, a slow robot shown faster than real under an honoured badge, and a predictive check that contradicted the live simulation, and only fixing both, and collapsing the four drifted kinematic replicas onto one gated shared model, made the fidelity claim true rather than aspirational. What I would do differently is to wire the feedback loop, and the single shared motion model, first rather than last, because both were the foundation the rest of the honesty rests on, and discovering their absence through review rather than by design cost rework.

9.3 Future work

The roadmap follows directly from the limitations, ordered roughly by the order the work should be taken on.

1. **Wire the imported specification through the Python engine**, teaching its sensor rays to honour an imported sensor's mount pose and range and the public API to read an imported mass, rpm and battery, so a measured build is simulated end to end on both engines rather than only in the studio. This closes the largest remaining fidelity-scope gap and is the natural next step after the formula parity that already landed.

2. **Implement and gate the remaining part commands** (the positioning sensor, the camera, the bumper and the gripper), exactly as the ultrasonic, inertial unit and line follower commands already are, so every fitted part offers a working, gated command, closing the gap between the parts catalogue and the runnable surface.
3. **Rigid-body physics on the hot path**, making the tested physics wrapper the source of truth for the visible motion, so the 3D Viewport reflects real contact forces, friction and collision response.
4. **A single shared arena definition** so the two engines agree on the world size and a program scores the same way it looked.
5. **Richer domain randomisation**, varying obstacle placement, lighting, friction and sensor noise across runs so that a passing program is one that generalises rather than one that memorised a layout.
6. **A source-cited real-world research mode** that, only after explicit opt-in, searches manufacturer datasheets and reputable suppliers for candidate parts, records retrieval dates and regions, and keeps electrical compatibility and purchasing decisions visibly unverified until checked by a competent person.
7. **An asset and interchange import workflow** on top of the specification import that already ships: a documented path for authoring parts and terrain, and a loader for standard robot descriptions such as URDF and for glTF geometry, so published robot models can be used directly rather than only their numbers.
8. **A research bridge** to the established heavyweight simulators, exporting a Kodro build and program for high-fidelity validation and pulling the results back, which keeps Kodro light while opening a path to rigorous physics when a user needs it.
9. **A bridge to a robotics middleware** as an optional, separate component that talks to a running daemon over a local socket, so a validated Kodro program can drive real topics, recorded honestly as a long way out because it fights the offline guarantee.
10. **The human pilot after ethics approval** specified in the evaluation, comparing deterministic evidence with the raw console for three diagnosis missions, followed only if justified by separate studies of classroom learning and refinement-memory efficacy.

9.4 Closing statement

The originality of this work is not a new algorithm. Its contribution is the implemented combination of specification-driven behaviour, disclosed fidelity, a lightweight simulation, build-aware generation and deterministic rejection in one accessible tool. A skeptical builder can compare design choices and export the assumptions behind the result, while a learner can acquire the programming and robotics concepts needed to test an idea. The strongest justified claim is that Kodro reduces uncertainty before purchase and catches defined classes of invalid program. Physical predictive validity, memory-driven improvement and usefulness to real users remain open evaluation questions.

Bibliography

Trinket (2026) *Trinket shutdown announcement*. Available at: https://trinket.io/announcement (Accessed: 27 July 2026).

Ahn, M., Brohan, A., Brown, N. et al. (2022) Do as I can, not as I say: grounding language in robotic affordances. *arXiv:2204.01691*. DOI: https://doi.org/10.48550/arXiv.2204.01691

Cemri, M., Pan, M.Z., Yang, S. et al. (2025) Why do multi-agent LLM systems fail? *arXiv:2503.13657*. DOI: https://doi.org/10.48550/arXiv.2503.13657

Chen, M., Tworek, J., Jun, H. et al. (2021) Evaluating large language models trained on code. *arXiv:2107.03374*. DOI: https://doi.org/10.48550/arXiv.2107.03374

dos Santos, V.G., Khadraoui, I., Farhat, I. et al. (2026) ALRM: agentic LLM for robotic manipulation. *arXiv:2601.19510*. DOI: https://doi.org/10.48550/arXiv.2601.19510

Gazebo (2026) *Gazebo Sim: a robotic simulator*. Available at: https://gazebosim.org/libs/sim/ (Accessed: 17 July 2026).

Hu, Z., Lucchetti, F., Schlesinger, C. et al. (2023) Deploying and evaluating LLMs to program service mobile robots. *arXiv:2311.11183*. DOI: https://doi.org/10.48550/arXiv.2311.11183 Published as open-source software at https://amrl.cs.utexas.edu/codebotler/.

Huang, L., Yu, W., Ma, W. et al. (2025) A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2), Article 42, pp. 1 to 55. DOI: https://doi.org/10.1145/3703155

Khati, D., Rodriguez-Cardenas, D., Pantzer, P. and Poshyvanyk, D. (2026) Detecting and correcting hallucinations in LLM-generated code via deterministic AST analysis. *arXiv:2601.19106*. DOI: https://doi.org/10.48550/arXiv.2601.19106

Lee, Y., Song, J.Y., Kim, D. et al. (2025) Hallucination by code generation LLMs: taxonomy, benchmarks, mitigation, and challenges. *arXiv:2504.20799*. DOI: https://doi.org/10.48550/arXiv.2504.20799

Lewis, P., Perez, E., Piktus, A. et al. (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, pp. 9459 to 9474. DOI: https://doi.org/10.48550/arXiv.2005.11401

Liang, J., Huang, W., Xia, F. et al. (2023) Code as policies: language model programs for embodied control. *2023 IEEE International Conference on Robotics and Automation*, pp. 9493 to 9500. DOI: https://doi.org/10.1109/ICRA48891.2023.10160591

Open Roberta (2026) *Open Roberta Lab*. Fraunhofer IAIS. Available at: https://lab.open-roberta.org/ (Accessed: 17 July 2026).

Papert, S. (1980) *Mindstorms: children, computers, and powerful ideas*. New York: Basic Books.

Reza, Z., Mazur, A., Dugdale, M.T. and Ray-Chaudhuri, R. (2025) Small models, big support: a local LLM framework for teacher-centric content creation and assessment using RAG and CAG. *arXiv:2506.05925*. DOI: https://doi.org/10.48550/arXiv.2506.05925

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. and Yao, S. (2023) Reflexion: language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36. Available at: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html (Accessed: 17 July 2026).

Sun, T., Xu, J., Li, Y. et al. (2025) BitsAI-CR: automated code review via LLM in practice. *arXiv:2501.15134*. DOI: https://doi.org/10.48550/arXiv.2501.15134

Tao, Z., Lin, T.E., Chen, X. et al. (2024) A survey on self-evolution of large language models. *arXiv:2404.14387*. DOI: https://doi.org/10.48550/arXiv.2404.14387

Tobin, J., Fong, R., Ray, A. et al. (2017) Domain randomization for transferring deep neural networks from simulation to the real world. *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 23 to 30. DOI: https://doi.org/10.1109/IROS.2017.8202133

VEX Robotics (2026) *Getting started with VEXcode VR*. Available at: https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr-blocks/introduction-and-fundamentals/lesson-1-getting-started-with-vexcode-vr (Accessed: 17 July 2026).

Wang, G., Xie, Y., Jiang, Y. et al. (2023) Voyager: an open-ended embodied agent with large language models. *arXiv:2305.16291*. DOI: https://doi.org/10.48550/arXiv.2305.16291

Wang, W., Li, Y., Jiao, L. and Yuan, J. (2026) Ro-SLM: onboard small language models for robot task planning and operation code generation. *arXiv:2604.10929*. DOI: https://doi.org/10.48550/arXiv.2604.10929

Wu, R., Wang, X., Mei, J. et al. (2025) EvolveR: self-evolving LLM agents through an experience-driven lifecycle. *arXiv:2510.16079*. DOI: https://doi.org/10.48550/arXiv.2510.16079

Cyberbotics (2026) *Webots user guide: foreword*. Available at: https://www.cyberbotics.com/doc/guide/foreword (Accessed: 17 July 2026).

Chapter A

Glossary and Abbreviations

Definitions are scoped to how each term is used in this dissertation.

Term	Meaning
API	Application programming interface. Here the small procedural command surface the user's program calls.
AST	Abstract syntax tree. Used by the sandbox to reject unsafe code and by the scorer to check required constructs.
Domain randomisation	Varying friction, mass, sensor noise and the scene across runs so behaviour that survives the spread is likely to survive reality.
Fidelity tier	One of honoured, approximated or not simulated. The disclosed level at which the simulation treats a reported figure.
Grounding	Supplying the active build and fitted-command set to generation. Deterministic validation separately rejects output that exceeds that set.
Kinematic motion	Motion advanced by a hand-written tick that fakes the feel of weight and traction, as distinct from a rigid-body solver that resolves forces.
KRS	Kodro Robot Specification. A schema-validated JSON document of a real robot's measured numbers (mass, drive geometry, battery, sensors) that can be imported or exported and that drives the simulation through the physical closed forms.
Ollama	A local model runtime. The optional, loopback-only backend for the assistant.
RobotSpec	The structured robot specification, imported as a KRS document or derived from chosen parts. The single source of truth for behaviour and the command registry.
Sandbox	The pre-execution check that rejects unsafe imports, builtins and operations before a program runs.
Shared motion model	The single set of closed-form equations and constants, mirrored in `motion-model.js` and `motion_model.py` and locked by conformance tests, that both engines obey.
SQLite	The embedded, file-based database used for the single local store.
Trace	The recorded sequence of command events from one run. The single representation that scoring, hinting and replay all read.

Table A.1. Glossary of terms as used in this dissertation.

Chapter B

The Command Surface and a First Program

The procedural command surface the user programs against is a small set of plainly named functions, gated by the build. The motion and query commands below are representative. A command is present only when the part it depends on is fitted.

```
move_forward(distance)       # drive forward. Metres in the bare style
move_backward(distance)      # drive backward
turn_left(degrees)           # turn in place to the left
turn_right(degrees)          # turn in place to the right
set_speed(percent)           # cap speed. Clamped by motor and mass
stop()                       # halt
read_distance()              # only if an ultrasonic or ranging sensor is fitted
read_heading()              # only if an inertial unit is fitted
collect_sample()             # only if a collector actuator is fitted
log(message)                 # write a line to the console
```

The program that ships loaded in the editor is the first example, and it teaches the habit the whole tool is built around: look before you move. It drives forward through the default city, and whenever the range sensor reports the way ahead is not clear it steers to a clearer path instead of driving into a parked car, then prints a short report of what happened. It calls only commands the default rover carries.

```
# Drive forward, but look first. distance() reads how far the way
# ahead is clear, in centimetres, so the rover steers around a
# parked car instead of driving into it.
set_speed(60)
for step in range(14):
    if distance() < 130:
        turn_right(55)
    else:
        move_forward(1)
```

Pressing run animates the rover along the path with weight transfer and a draining battery. Pressing step advances one event at a time. Changing the build, a heavier chassis or a weaker motor, changes how the same program behaves, which is the point the whole platform is built to make.